Appendix R10: Decision Trees

Bagging, Random Forest & Boosted Trees

J. Alberto Espinosa

2024-06-22

This Quarto document contains R code examples about decision trees, which can be of two types, regression trees and classification trees. Regular trees are generally not very accurate and not that great for interpretability, so other models are generally preferred to trees, although they can help prepare useful visuals for managers. However, there are several advance predictive modeling methods based on trees, which take advantage of things like re-sampling, re-fitting, randomizing, boosting, etc., making some of these advanced tree models very accurate and interpretable. We start by discussing how to identify the optimal tree size and evaluate accuracy using cross-validation. We then discuss a few very popular advanced tree methods, including Bootstrap Aggregation (BAgging), Random Forest and Boosted Trees. I illustrate these methods for both, regression and classification models. For the corresponding conceptual material associated with these appendinces, consult the main book Predictive Analytics and Machine Learning for Managers.

Table of contents

Decision Trees

I covered basic regression and classification trees in the R appendix to Chapter 3. To follow this appendix, it is important to review the material and appendix on trees in Chapter 3. In that appendix, I illustrated how to fit regression and classification trees, and also how to control the size of the tree with the `mindev =` parameter. However, this parameter only restricts the size of the tree, but does not let us identify the optimal tree size. As I discussed earlier, plain trees are generally not as accurate as other predictive models. However, you can improve the performance of tree models by finding the optimal size of a tree using cross-validation. In addition, there are advanced tree models that have proven to be quite accurate, sometimes outperforming other regression models. In this appendix, I illustrate how to identify the optimal size of a tree using cross-validation, and also discuss three advanced tree modeling methods: **Bootstrap Aggregation (Bagging)**, **Random Forest** and **Boosted Trees**.

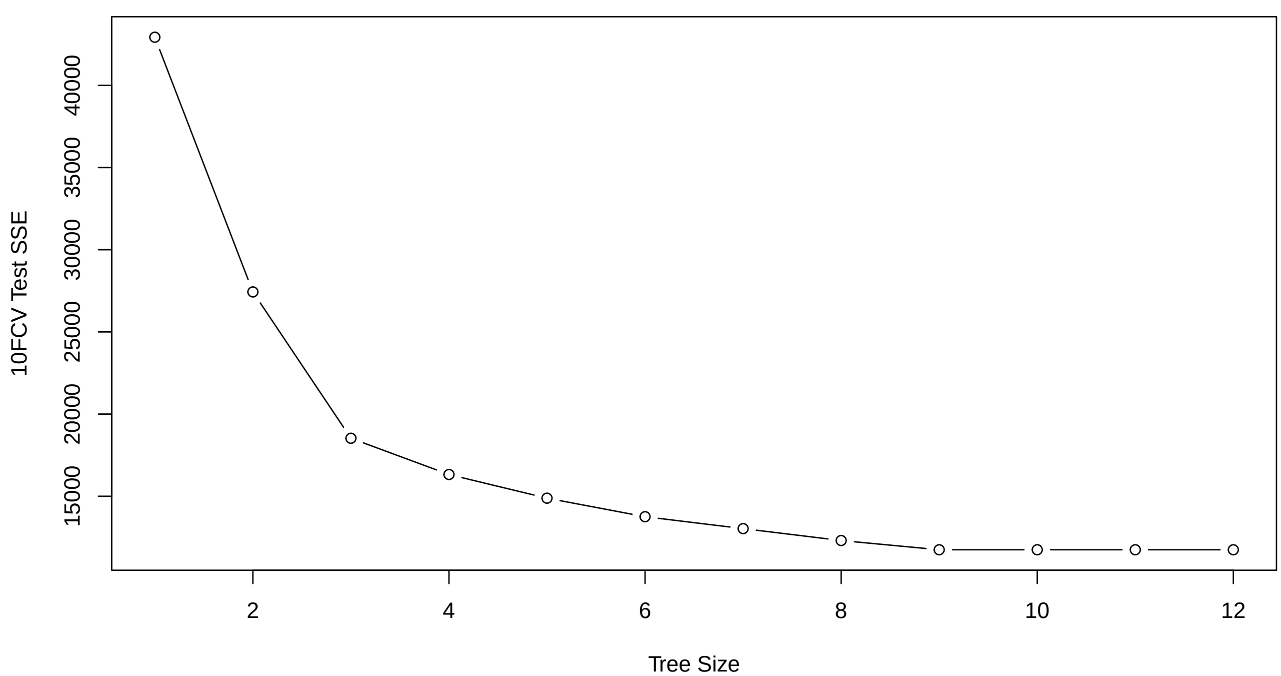
Optimal Regression Tree Size with Cross-Validation

Cross validation of tree results. Let's explore the CV for various pruned trees. I use the `cv.tree()` function from the `{tree}` library. I illustrate this example using the **Boston** housing data set from the `{MASS}` library.

```
library(MASS) # Contains the Boston data set
library(tree)
set.seed(1) # Arbitrary, to get repeatable results

tree.boston <- tree(medv ~ ., Boston, mindev = 0.005) # Fit the tree
cv.boston <- cv.tree(tree.boston) # Extract CV data from it
```

Plotting the tree size (number of terminal nodes) against deviance:

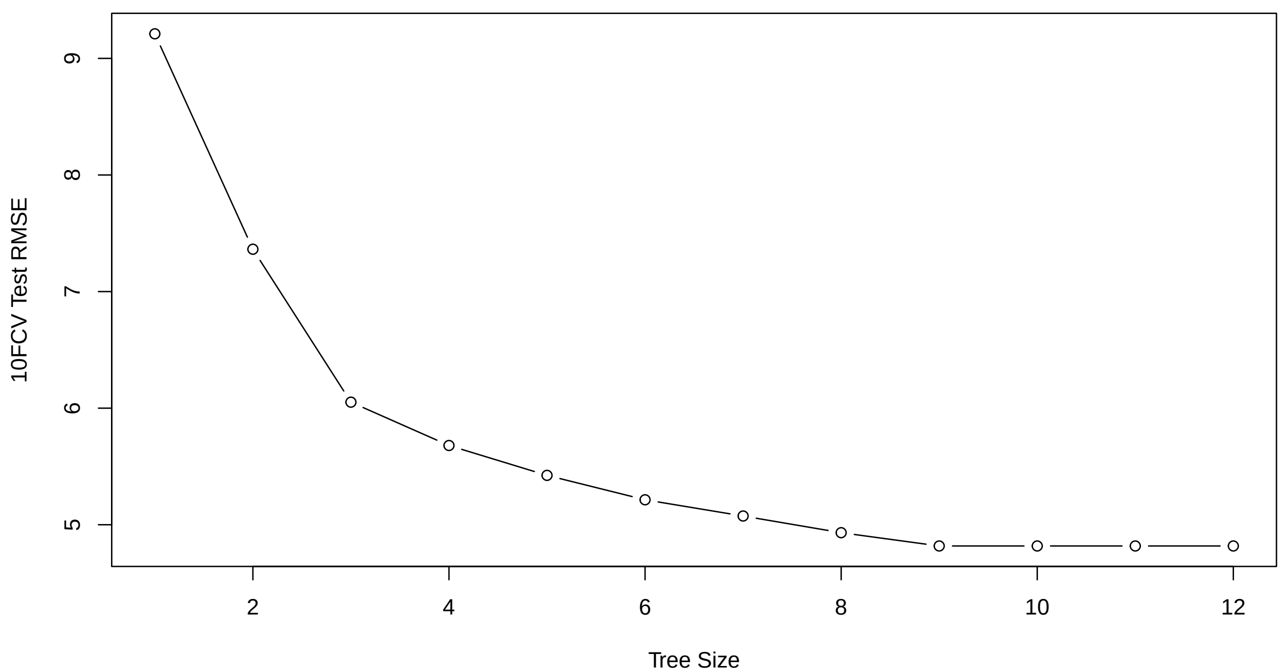
```
plot(cv.boston$size,
     cv.boston$dev,
     xlab = "Tree Size",
     ylab = "10FCV Test SSE",
     type = 'b')
```



Notes:

- The `cv.tree()` function does **10FCV** and minimizes the deviance measured as the MSE in regression trees (or 2LL for classification trees). You can change the 10-Fold to other folds with the attribute `K =`.
- `type = 'b'` is for **"both"**, points and lines
- The `$dev` value in the plot is the sum of squared errors (SSE), which we can easily convert to MSE by dividing by the number of observations `nrow(Boston)`. I also convert the MSE values into RMSE by taking the `sqrt()` to get smaller and more meaningful values.

```
cv.boston.rmse <- sqrt(cv.boston$dev / nrow(Boston))
plot(cv.boston$size,
     cv.boston.rmse,
     xlab = "Tree Size",
     ylab = "10FCV Test RMSE",
     type = 'b')
```



Let's now list the tree size and their corresponding deviance:

```
round(
  cbind("Size" = cv.boston$size,
        "10FCV Test RMSE" = cv.boston.rmse,
        "Test SSE" = cv.boston$dev),
  digits = 2)
```

```
     Size 10FCV Test RMSE Test SSE
[1,]   12            4.82 11744.91
[2,]   11            4.82 11744.91
[3,]   10            4.82 11744.91
```

```
 [4,]    9             4.82 11744.91
 [5,]    8             4.93 12306.70
 [6,]    7             5.07 13031.50
 [7,]    6             5.21 13755.94
 [8,]    5             5.42 14883.37
 [9,]    4             5.68 16323.27
[10,]    3             6.05 18528.82
[11,]    2             7.36 27433.88
[12,]    1             9.21 42930.57
```

Since the data is in 2 vectors, we can find the best tree size computationally:

```
min.sse <- min(cv.boston$dev) # smallest deviance SSE
min.rmse <- round(min(cv.boston.rmse), digits = 2)
best.ind <- which(cv.boston$dev == min.sse) # index of best tree
best.size <- cv.boston$size[best.ind] # Find best size
```

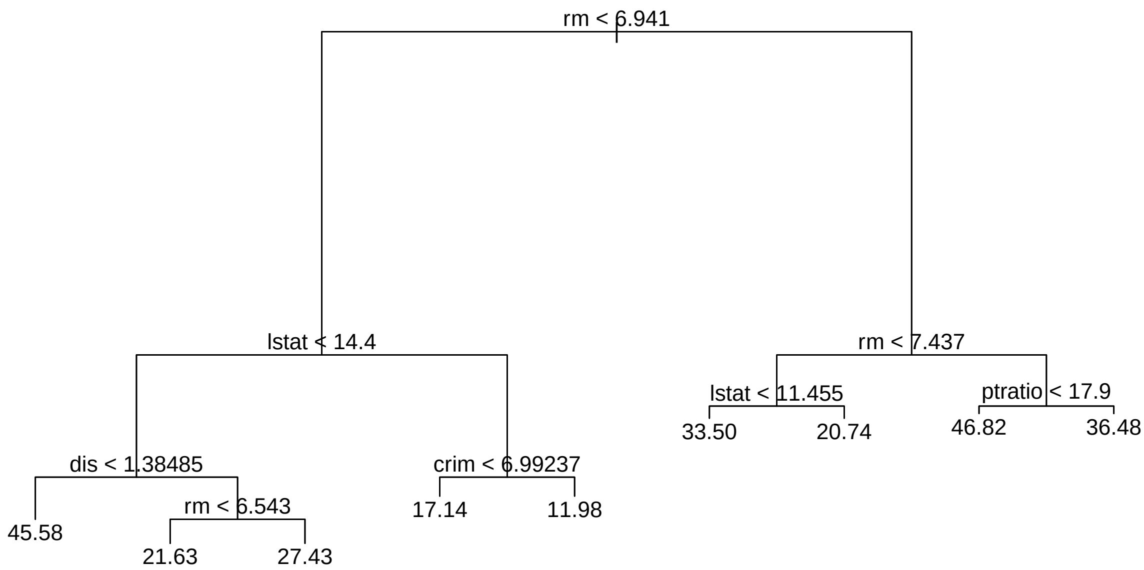
List all three values

```
cbind("Smallest Deviance" = min.sse,
      "Smallest CV rMSE" = min.rmse,
      "Tree Index" = best.ind,
      "Tree Size" = best.size)
```

```
     Smallest Deviance Smallest CV rMSE Tree Index Tree Size
[1,]          11744.91             4.82          1        12
[2,]          11744.91             4.82          2        11
[3,]          11744.91             4.82          3        10
[4,]          11744.91             4.82          4         9
```

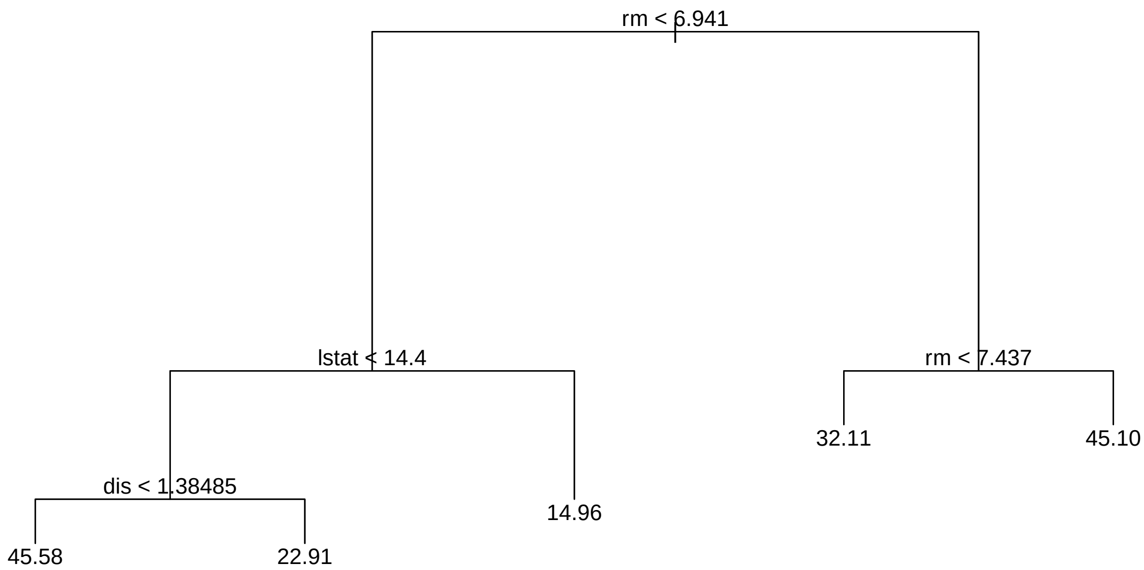
Notice that we were asking for the best CV deviance, but we got 4 trees instead. This happened because of a 4-way tie in deviance. In this case, we chose the smallest tree of the 4 which is the one with **9 branches**. In case of a tie, it is best to use the smallest and more parsimonious tree. Since **best.size** is a vector in this case and we need a scalar value, let's just pick the smallest size of the 4 best and then prune the tree using this **best.size**.

```
best.size <- min(best.size) # Select smallest tree if there is a tie
prune.boston <- prune.tree(tree.boston,
                           best = best.size)
plot(prune.boston)
text(prune.boston, pretty = 0)
```



Just to illustrate, let's prune to just 5 terminal nodes (or regions, or leaves):

```
prune.boston <- prune.tree(tree.boston,
                           best = 5)
plot(prune.boston)
text(prune.boston, pretty = 0)
```



Random Sample CV with Trees

If instead of using `cv.tree` to do cross-validation, if you prefer to do sub-sample splitting manually, you can do so with training and test data (illustrated with the Boston data set):

```
set.seed(1) # To get replicable results
trsize <- 0.7
train <- sample(1:nrow(Boston),
                trsize * nrow(Boston)) # Train index
Boston.train <- Boston[train, ] # Train sub-sample
Boston.test <- Boston[-train, ] # Test sub-sample
```

Now fit a regression tree on the train data

```
tree.boston.tr <- tree(medv ~ ., Boston.train)
```

Then use the trained model to test it with the test subset

```
pred.medv <- predict(tree.boston.tr, Boston.test)
```

Now let's compute the **CV Test MSE**

```
mse.tree <- mean( (pred.medv - Boston.test$medv) ^ 2 )
mse.tree
```

```
[1] 36.2319
```

The disadvantage of finding the CV Test MSE this way is that you need to fit multiple trees with a loop or else to identify the best tree. The `cv.tree()` approach is more efficient.

Classification Tree Size with Cross-Validation

Let's fit a classification tree with the **diamonds** data set in the `{ggplot2}` library. But instead of predicting price, let's divide the diamonds into expensive ones if their price is over the median price, and affordable (i.e., not expensive) otherwise.

```
library(ggplot2)
median.price <- median(diamonds$price)
diamonds$expensive <- factor(ifelse(diamonds$price <= median.price,
                              "No", "Yes"))
table(diamonds$expensive) # Roughly evenly divided
```

```
   No   Yes
26985 26955
```

Let's now fit a classification tree to predict what makes diamonds expensive, using a few predictors and then extract the CV data. Let's use a small mindev to get a larger tree.

Note: `FUN = prune.misclass` parameter uses misclassification error for cross-validation and pruning. Otherwise, the default is deviance.

```
diamonds.exp <- tree(expensive ~ carat + cut + color + clarity,
                     data = diamonds,
                     mindev = 0.001)
summary(diamonds.exp)
```

```
Classification tree:
tree(formula = expensive ~ carat + cut + color + clarity, data = diamonds,
    mindev = 0.001)
Number of terminal nodes:  28
Residual mean deviance:  0.134 = 7224 / 53910
Misclassification error rate: 0.02842 = 1533 / 53940
```

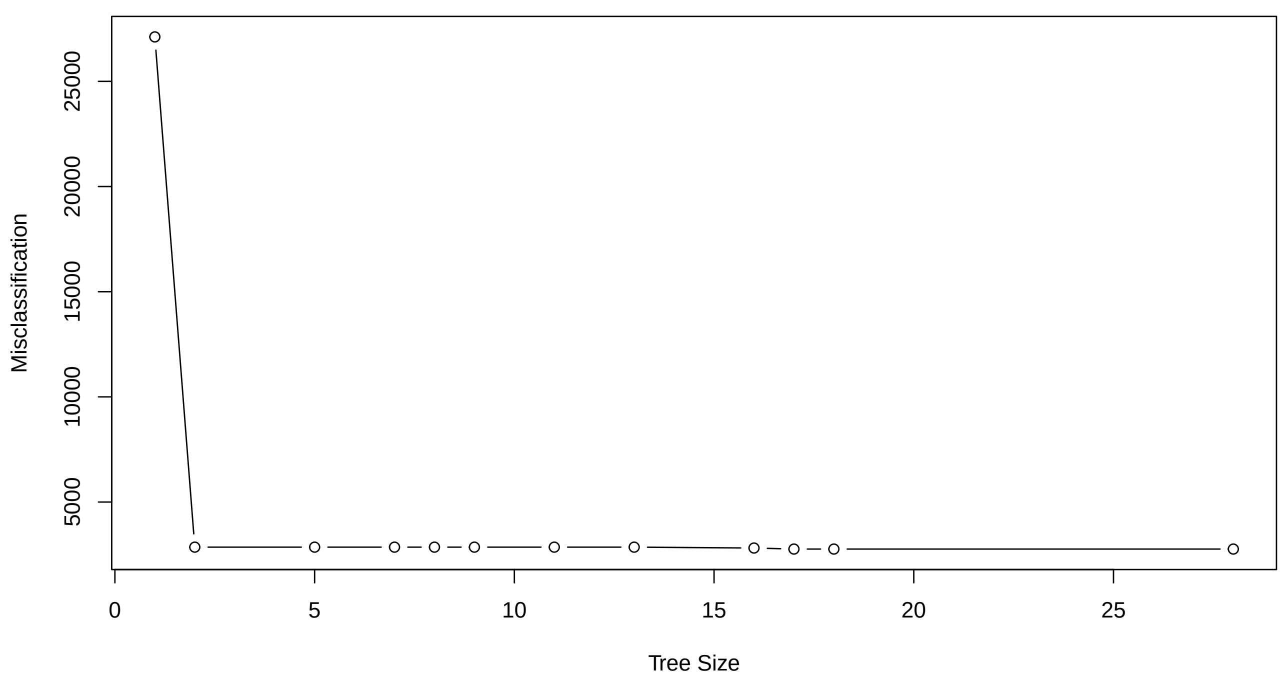
- The **Number of Terminal Nodes** is the number of branches at the bottom end of the tree due to either the resulting tree size from the mindev parameter, or the tree size when all branches are pure (i.e., can no longer continue splitting the tree).
- The **residual mean deviance** in classification trees is the **total residual deviance** (i.e., the sum of squares of the residuals) divided by the (total number of observations - number of terminal nodes). This statistic is **not as useful** with classification trees (as it is for regression trees), as the sum of squares of the residual is not as meaningful. However, they it is useful to compare various tree models -> the model with the smallest deviance (between actual and predicted values is better).
- The **mis-classification error** is more meaningful. It is the total number of **mis-classified observations**, relative to the **total** number of **observations**.

These are useful statistics, but it is much better to compare trees using cross-validation, as I do next.

```
set.seed(1)

cv.diamonds <- cv.tree(diamonds.exp,
                       FUN = prune.misclass)

cv.diamonds
```

```
$size
 [1] 28 18 17 16 13 11  9  8  7  5  2  1

$dev
 [1]  2765  2765  2765  2814  2856  2856  2856  2856  2856  2856  2856 27114

$k
 [1]        -Inf     0.00000    20.00000    26.00000    40.66667    53.50000
 [7]    55.50000    59.00000    91.00000    95.00000   199.00000 24099.00000

$method
[1] "misclass"

attr(,"class")
[1] "prune"         "tree.sequence"
```

```r
plot(cv.diamonds$size,
     cv.diamonds$dev,
     type = 'b',
     xlab = "Tree Size",
     ylab = "Misclassification")
```




```r
cbind("Tree Size" =  cv.diamonds$size,
      "Misclassification" = cv.diamonds$dev)
```

```
     Tree Size Misclassification
 [1,]        28              2765
```

```
 [2,]        18             2765
 [3,]        17             2765
 [4,]        16             2814
 [5,]        13             2856
 [6,]        11             2856
 [7,]         9             2856
 [8,]         8             2856
 [9,]         7             2856
[10,]         5             2856
[11,]         2             2856
[12,]         1            27114
```

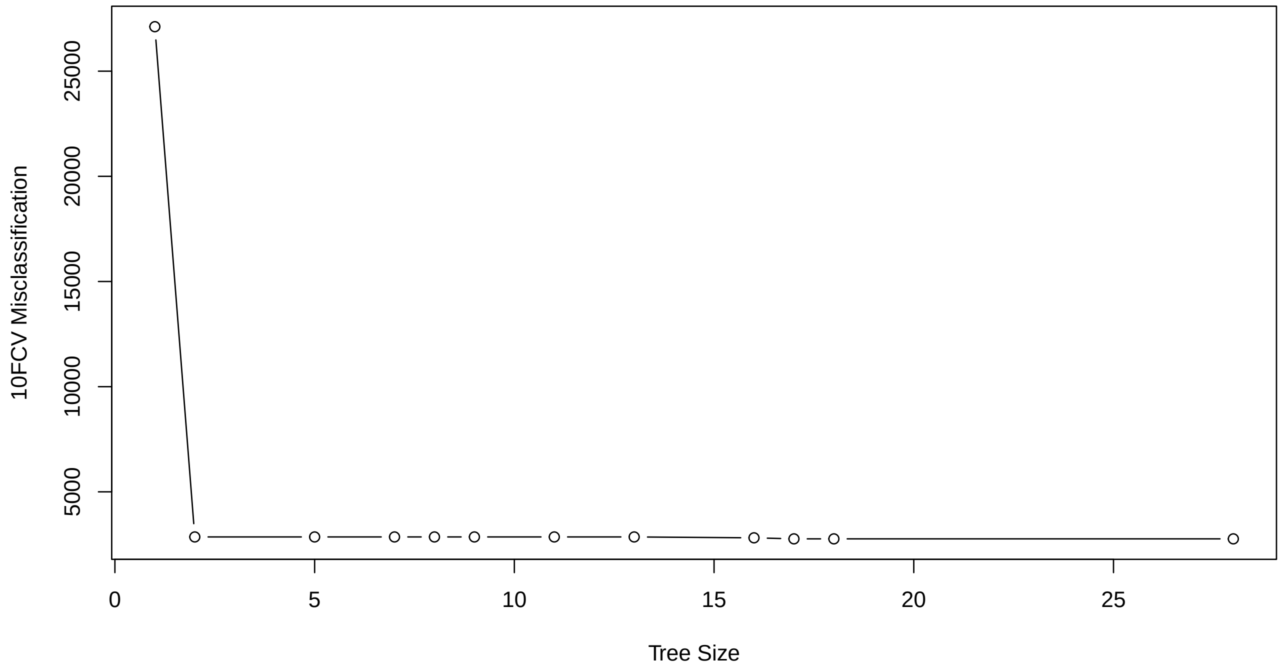
`$size` and `$dev` report the tree size and the corresponding CV miss-classification rate in this case.

```r
cbind("Tree Size" = cv.diamonds$size,
      "Misclass" = cv.diamonds$dev)
```

```
      Tree Size Misclass
 [1,]        28     2765
 [2,]        18     2765
 [3,]        17     2765
 [4,]        16     2814
 [5,]        13     2856
 [6,]        11     2856
 [7,]         9     2856
 [8,]         8     2856
 [9,]         7     2856
[10,]         5     2856
[11,]         2     2856
[12,]         1    27114
```

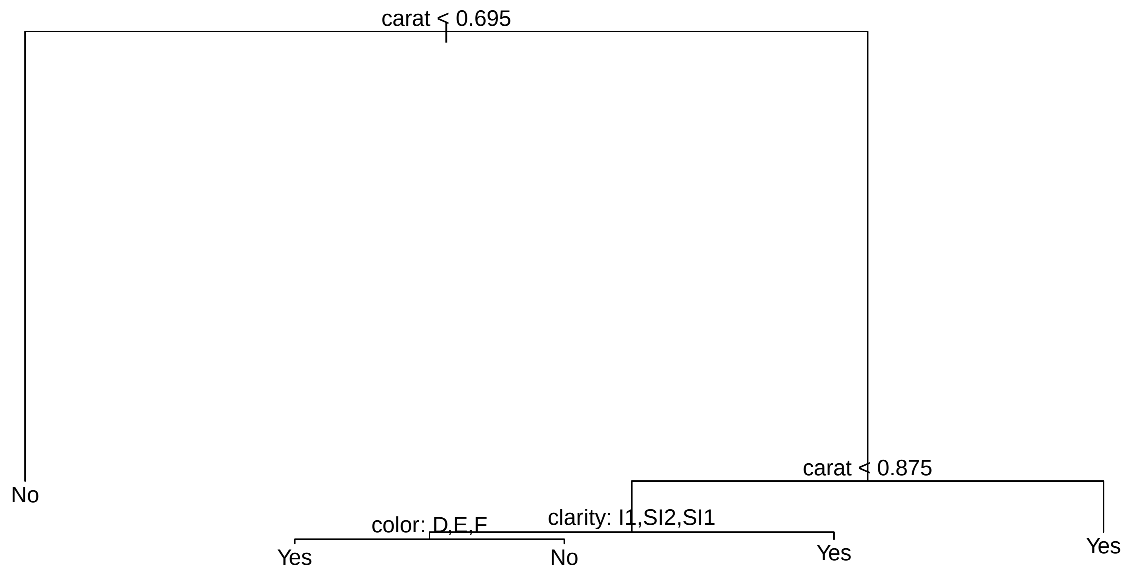
Let's inspect the trees visually. Let's plot CV for each tree size, misclassification in this case.

```r
plot(cv.diamonds$size,
     cv.diamonds$dev,
     xlab = "Tree Size",
     ylab = "10FCV Misclassification",
     type = "b") # type="b" plots "both", lines and dots
```



Misclassification is lowest at 28 nodes, but flattens very quickly at around 2, so let's prune to 5 nodes.

```
prune.diamonds <- prune.misclass(diamonds.exp,
                                 best = 5)
plot(prune.diamonds) # Plot the tree
text(prune.diamonds, pretty = 0) # With labels
```



Tree splitting based on Gini index of Purity

The default leaf splitting method is **misclassification**. But you can split leaves using the **Gini** index of **impurity**. Notice that the misclassification error in this case improves with impurity splitting. This is not always the case. If the data has high classification purity, then `split = "gini"` will work better. Otherwise the default based on miss-classification will work best. The best thing is to try both.

```
diamonds.exp.g <- tree(expensive ~ carat + cut + color + clarity,
                       data = diamonds,
                       mindev = 0.001,
                       split = "gini")
summary(diamonds.exp.g) # Notice the higher misclassification error
```

```
Classification tree:
tree(formula = expensive ~ carat + cut + color + clarity, data = diamonds,
    split = "gini", mindev = 0.001)
Number of terminal nodes:  582
Residual mean deviance:  0.07269 = 3878 / 53360
Misclassification error rate: 0.01674 = 903 / 53940
```

Random Split CV of Trees with the Confusion Matrix

The method above fits a tree model with the entire data, but then the cv.tree() function does 10-Fold Cross Validation to find the optimal tree size to prune to. This is good if you are only interested in overall predictive accuracy. But in most classification models (e.g., logistic, LDA, etc.), we are also interested in accuracy statistics like **sensitivity**, **specificity** and **false positives**. We can compute all the necessary **Confusion Matrix** statistics exactly as we did for classification models. We use random splitting CV to build the Confusion Matrix and compute the metrics. Let's illustrate an example with the **Heart.csv** data set.

Note: I use the file **Heart.csv**, which I pre-downloaded, but you can access the data set from the `{ISLR}` book authors and library developer's site as follows:

```
heart <- read.table("https://www.statlearning.com/s/Credit.csv",
                    sep = ",",
                    header = T,
                    stringsAsFactors = T)
```

```
heart <- read.table("Heart.csv",
                    sep = ",",
                    header = T,
                    stringsAsFactors = T)

heart$chd <- as.factor(heart$chd) # Convert chd from numeric to factor
class(heart$chd) # Confirming that chd is a factor variable
```

```
[1] "factor"
```

Let's now extract the train and test subsets

```
set.seed(1) # Arbitrary
trsize <- 0.7
train <- sample(1:nrow(heart),
                trsize * nrow(heart))
heart.train <- heart[train, ] # Train subset
heart.test <- heart[-train, ] # Test subset
```

Train the tree with the train subset, and then make classification predictions using the trained model with the test subset. We use the **predict()** to predict **classifications** (0 or 1) using the parameter `type = "class"`. If the probability of an outcome is greater than **0.5** the predicted classification is 1, and 0 otherwise. Note that I'm naming the prediction object **heart.tree.pred.class.5**. It's a long name, but it helps to remember that it's tree prediction for the heart data set, with classification outcomes with a threshold of 0.5. The long name makes it easier to remember all this.

```
heart.tree.train <- tree(chd ~ .,
                         data = heart.train)
heart.tree.pred.class.5 <- predict(heart.tree.train, heart.test,
                                   type = "class")
```

If we want to use a different classification threshold, we need to make probability predictions and convert them to classifications with an `ifelse()` function, as I illustrate a bit later. If we omit the parameter `type = "class"`, the default prediction is the probability of the outcome being 1.

```
heart.tree.pred.prob <- predict(heart.tree.train, heart.test)
```

Confusion Matrix

```
conf.mat <- table("Predicted" = heart.tree.pred.class.5,
                  "Actual" = heart.test$chd)

colnames(conf.mat) <- rownames(conf.mat) <- c("No","Yes")

conf.mat <- addmargins(conf.mat,
                       margin = c(1, 2),
```

```
                          FUN = sum) # Add margin totals
```

```
Margins computed over dimensions
in the following order:
1: Predicted
2: Actual
```

```
conf.mat # Take a look
```

```
         Actual
Predicted  No Yes sum
      No   68  28  96
      Yes  22  21  43
      sum  90  49 139
```

Confusion Matrix Metrics

```
TruN <- conf.mat[1, 1] # True negatives
TruP <- conf.mat[2, 2] # True positives
FalN <- conf.mat[1, 2] # False negatives
FalP <- conf.mat[2, 1] # False positives

TotN <- TruN + FalP # Total actual negatives
TotP <- TruP + FalN # Total actual positives
TotNpr <- TruN + FalN # Total negative predictions
TotPpr <- TruP + FalP # Total positive predictions
Tot <- TotN + TotP # Total

cbind(TruN, TruP, FalN, FalP, TotN, TotP, TotNpr, TotPpr, Tot)
```

```
     TruN TruP FalN FalP TotN TotP TotNpr TotPpr Tot
[1,]   68   21   28   22   90   49     96     43 139
```

Confusion Matrix Accuracy and Error Rates

```
Accuracy.Rate <- (TruN + TruP) / Tot
Error.Rate <- (FalN + FalP) / Tot
Sensitivity <- TruP / TotP
Specificity <- TruN / TotN
FalseP.Rate <- 1 - Specificity

heart.tree.rates.5 <- c(Accuracy.Rate, Error.Rate,
                        Sensitivity, Specificity,
                        FalseP.Rate)
names(heart.tree.rates.5) <- c("Accuracy", "Error",
                               "Sensitivity", "Specificity",
                               "False Pos")
print(heart.tree.rates.5, digits = 2)
```

```
   Accuracy       Error Sensitivity Specificity   False Pos
       0.64        0.36        0.43        0.76        0.24
```

If we want to change the **classification threshold** to, say `0.6`, you can compute the classifications as follows:

```r
thresh <- 0.6 # Set the threshold
heart.tree.pred.class.6 <- ifelse(heart.tree.pred.prob[, 2] > thresh,
                                  1, 0)
conf.mat <- table("Predicted" = heart.tree.pred.class.6,
                  "Actual" = heart.test$chd)

colnames(conf.mat) <- rownames(conf.mat) <- c("No","Yes")

conf.mat <- addmargins(conf.mat,
                       margin = c(1, 2),
                       FUN = sum) # Add margin totals
```

```
Margins computed over dimensions
in the following order:
1: Predicted
2: Actual
```

```r
conf.mat
```

```
         Actual
Predicted  No Yes sum
      No   73  33 106
      Yes  17  16  33
      sum  90  49 139
```

The rest of the confusion matrix script is exactly the same as above:

```r
TruN <- conf.mat[1,1] # True negatives
TruP <- conf.mat[2,2] # True positives
FalN <- conf.mat[1,2] # False negatives
FalP <- conf.mat[2,1] # False positives

TotN <- TruN + FalP # Total actual negatives
TotP <- TruP + FalN # Total actual positives
TotNpr <- TruN + FalN # Total negative predictions
TotPpr <- TruP + FalP # Total positive predictions
Tot <- TotN + TotP # Total

cbind(TruN, TruP, FalN, FalP, TotN, TotP, TotNpr, TotPpr, Tot)
```

```
     TruN TruP FalN FalP TotN TotP TotNpr TotPpr Tot
[1,]   73   16   33   17   90   49    106     33 139
```

Confusion Matrix Accuracy and Error Rates

```r
Accuracy.Rate <- (TruN + TruP) / Tot
Error.Rate <- (FalN + FalP) / Tot
Sensitivity <- TruP / TotP
Specificity <- TruN / TotN
FalseP.Rate <- 1 - Specificity

heart.tree.rates.6 <- c(Accuracy.Rate, Error.Rate,
```

```
                         Sensitivity, Specificity,
                         FalseP.Rate)
names(heart.tree.rates.6) <- c("Accuracy", "Error",
                               "Sensitivity", "Specificity",
                               "False Pos")
print(heart.tree.rates.6, digits = 2)
```

```
   Accuracy       Error Sensitivity Specificity   False Pos
       0.64        0.36        0.33        0.81        0.19
```

Both thresholds together

```
heart.tree.rates.both <-
    round(rbind(heart.tree.rates.5,
                heart.tree.rates.6),
          digits = 3)

rownames(heart.tree.rates.both) <- c("50% Threshold",
                                     "60% Threshold")

heart.tree.rates.both # Check it out
```

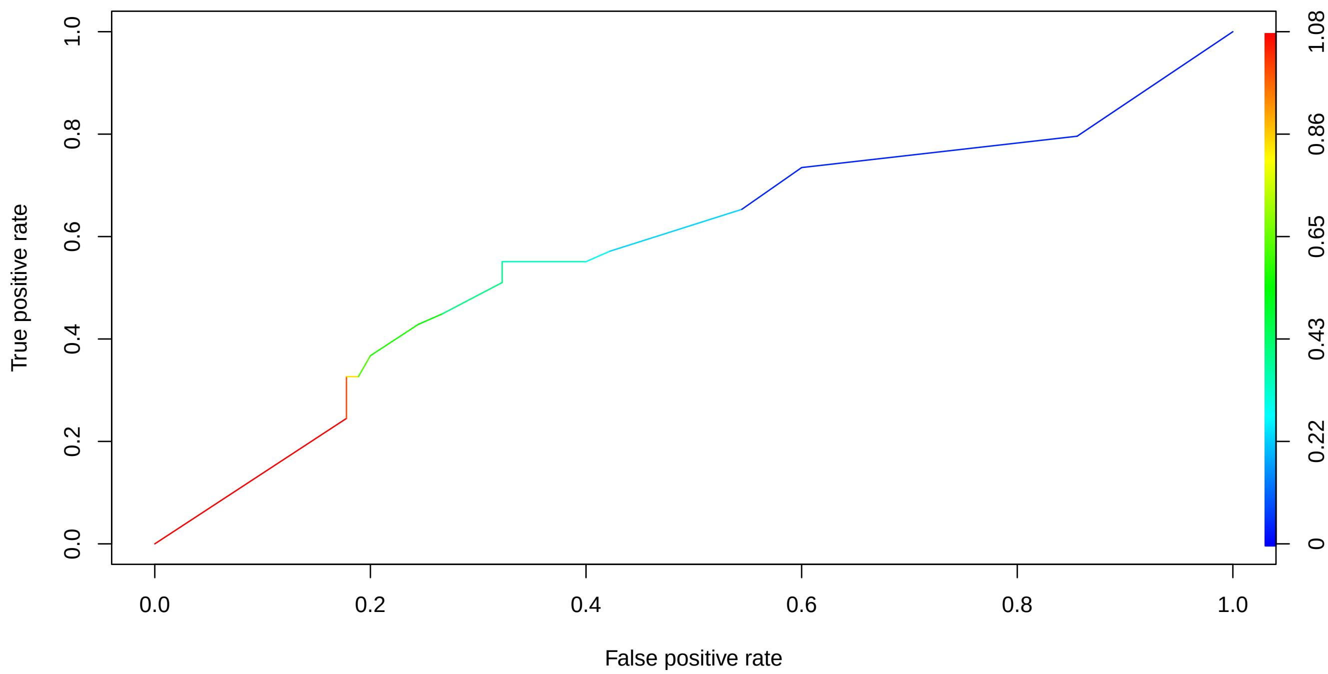
```
              Accuracy Error Sensitivity Specificity False Pos
50% Threshold     0.64  0.36       0.429       0.756     0.244
60% Threshold     0.64  0.36       0.327       0.811     0.189
```

Based on the results above, can you figure out which model is better overall? When predicting positives? When predicting Negatives?

ROC Curves for Classification Trees

For ROC curves, we use the `prediction()` and `performance()` functions from the `{ROCR}` library, exactly as we did for other classification models. We use the **predicted probabilities** `heart.tree.pred.prob` that we computed above. This object is matrix with 2 columns. The first has the probability of being 0 and the second has the probability of being 1. We only need the second column, which we can extract with the [, 2] index.

```
library(ROCR)
pred <- prediction(heart.tree.pred.prob[, 2],
                   heart.test$chd)
perf <- performance(pred, "tpr", "fpr")
plot(perf, colorize = T)
```




```
auc <- performance(pred, "auc") # Compute the AUC
auc.name <- auc@y.name[[1]] # AUC label text
auc.value <- round(auc@y.values[[1]], digits = 3) # AUC value, rounded
paste(auc.name, "is", auc.value) # The model is not so great
```

```
[1] "Area under the ROC curve is 0.577"
```

Bootstrap Aggregation (Bagging) Trees

Trees are notoriously inaccurate. Part of the issue is that the first split can totally change the results. So, it is not uncommon to have trees with both, bias and variance. But as it turns out, if you fit many trees, they tend to be less biased and more accurate on average. Bagging, Random Forest and Boosted trees take advantage of this property, rendering more accurate results.

Bagging stands for **Bootstrap Aggregation**. Bootstrap is the process of drawing multiple random samples of data to fit a model, but the samples are drawn with replacement. If the data set has n observations, the bootstrap sample matches that by drawing n observations. But because the sample is drawn with replacement, some observations will be repeated and some will be left out. The idea with bootstrapping is by drawin hundreds or thousands of bootstrap samples, in the end, all data points will be represented.

With **Bagging**, each bootstrapped tree includes **all predictors** in the model, fitted with a different bootstrap sample. The results are then aggregated, reducing both, bias and variance, yielding more accurate and stable results than single trees. The main **tuning parameters** in Bagging is the size of the random sample, which is generally the same as the data set size, and the **number of trees** bootstrapped, fitted and aggregated.

As I discuss a bit later, **Bagging** is a special case of **Random Forest**. So, we use the `randomForest()` function from the `{randomForest}` library.

```
library(randomForest)
library(MASS) # Contains the Boston housing data set
```

Let's now fit a Bagging tree model. Note that this is a regression tree model, but the same syntax applies to classification trees. A **Bagging** tree is the same as a **Random Forest** tree, but the bagging tree includes the same predictors in each bootsrapped tree. We will see later that this is not the case in more general Random Forest trees.

As you can see below, there are 13 predictors in the model, so we use `mtry = 13` to include all 13 predictors in each bootstrapped tree. The parameter `importance = T` instructs the `randomForest()` function to compute the **variable importance** graphs, which I discuss a bit later. I also compute the **10FCV RMSE** by taking the `sqrt()` of the MSE.

```
ncol(Boston) # 14 variables -> 1 outcome and 13 predictors
```

```
[1] 14
```

```
bag.boston <- randomForest(medv ~ .,
                           data = Boston,
                           mtry = 13,
                           importance = T)

bag.boston # Take a quick look.
```

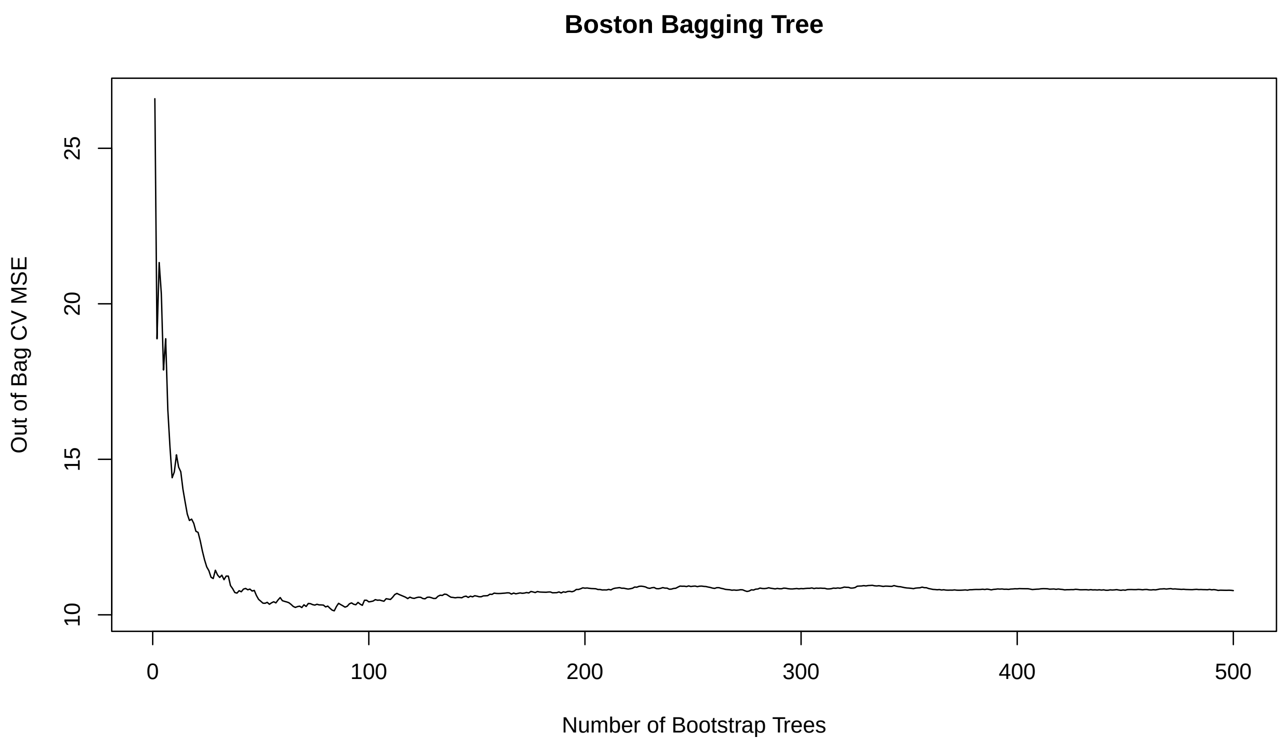
```

Call:
 randomForest(formula = medv ~ ., data = Boston, mtry = 13, importance = T)
               Type of random forest: regression
                     Number of trees: 500
No. of variables tried at each split: 13

          Mean of squared residuals: 10.7787
                    % Var explained: 87.23
```

```
cat("\n", "10FCV RMSE =", sqrt(mean(bag.boston$mse)), "\n")
```

```

 10FCV RMSE = 3.313163
```

The error is the out of bag MSE averaged over all bootstrapped trees trees. To see how the out of bag MSE changes as the number of bootstrapped trees increase see:

```
plot(bag.boston$mse,
     main = "Boston Bagging Tree",
     xlab = "Number of Bootstrap Trees",
     ylab = "Out of Bag CV MSE",
     type="l")
```

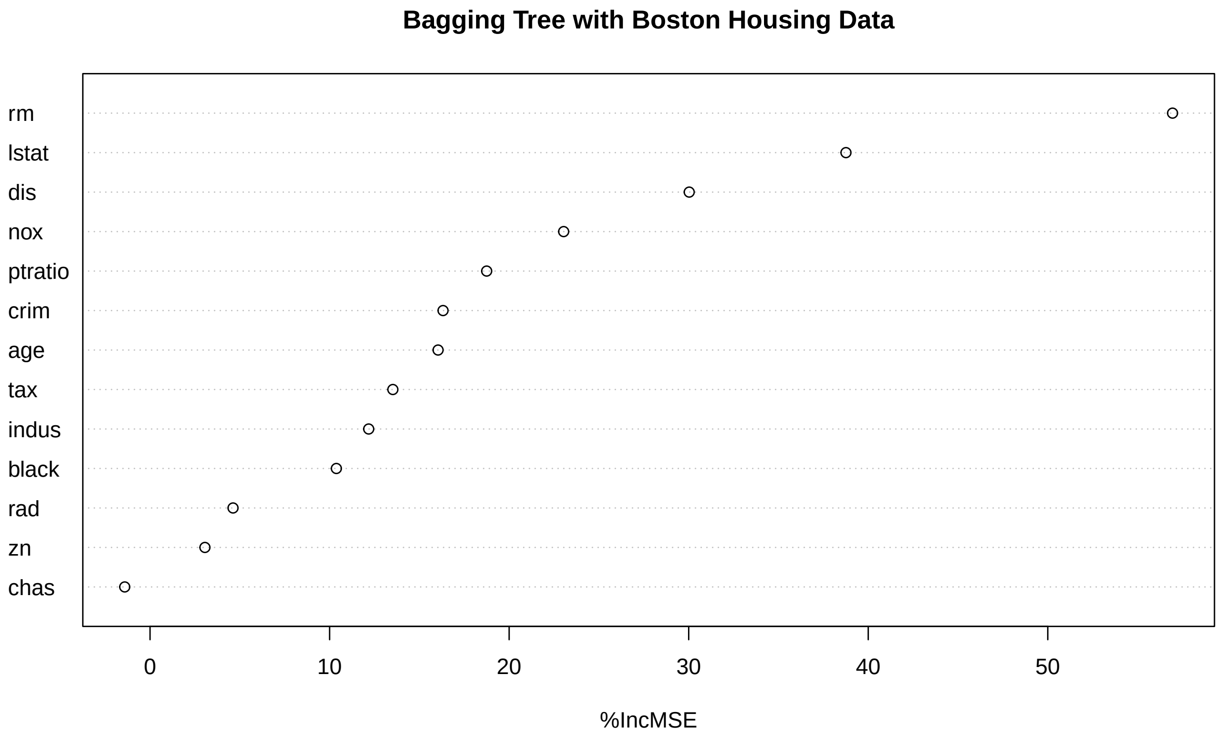


One shortcoming of trees is that there are no coefficients or p-values to ascertain which variables have stronger effects. But the **variable importance** computations help overcome this problem by displaying the importance of each variable to the tree model.

Variable Importance Plot. Note: `type = 1` renders variable importance in terms of % contribution to **MSE reduction**, which is the type to use for quantitative models. Use `type = 2` for **purity** improvement in classification models.

Variable Importance in BAgging trees

```
varImpPlot(bag.boston, # Variable importance plot
           type = 1,
           main = "Bagging Tree with Boston Housing Data")
```




```r
round(importance(bag.boston,
                 type = 1),
      digits = 2) # Variable importance values
```

```
        %IncMSE
crim      16.32
zn         3.06
indus     12.18
chas      -1.41
nox       23.04
rm        56.95
age       16.04
dis       30.03
rad        4.62
tax       13.52
ptratio   18.75
black     10.38
lstat     38.76
```

The 2 values of reported for variable importance are:

- `type = 1` for **regression trees**: mean increase (over all trees) in accuracy **(% MSE explained)** when the variable is added to the the model
- `type = 2` for **classification trees**: mean increase (over all trees) in **purity** when the tree is split by that variable

Higher values are always best for types

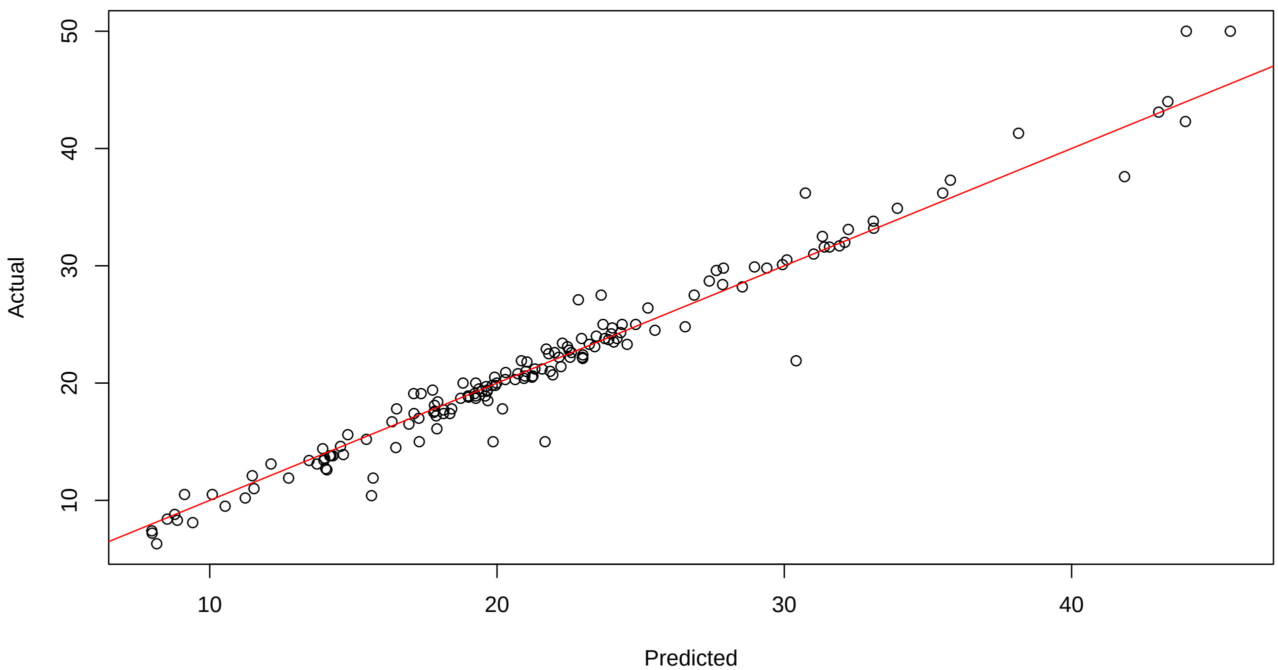
Bagging Tree Predictions and Cross-Validation

Let's re-create the Boston training subset we created above for easy reference.

```
set.seed(1) # To get repeatable results
trsize <- 0.7

train <- sample(1:nrow(Boston),
                trsize * nrow(Boston))
Boston.train <- Boston[train, ] # Train sub-sample
Boston.test <- Boston[-train, ] # Test sub-sample
```

Let's make predictions with the trained model and the test subset, and plot the predictions against the actual values, to evaluate the accuracy of our predictions.

```
bag.pred <- predict(bag.boston, newdata = Boston.test)
plot(bag.pred,
     Boston.test$medv,
     xlab = "Predicted",
     ylab = "Actual") # Plot pred vs. actual
abline(0, 1, col = "red") # 45 degree line (intercept = 0; slope = 1)
```



Now let's calculate the CV Test MSE

```
mse.bag <- mean((bag.pred - Boston.test$medv) ^ 2)
mse.bag
```

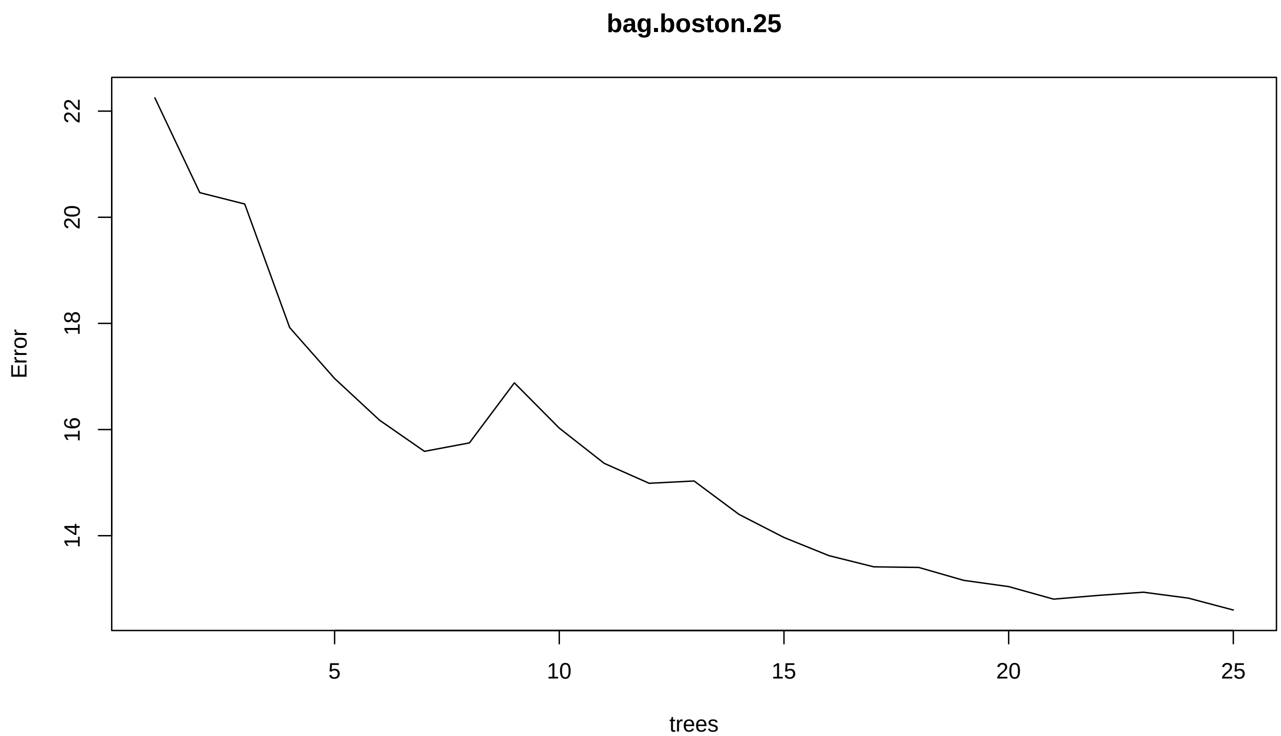
```
[1] 2.841752
```

Notice that the MSE is almost 1/2 of the regression tree MSE.

Technical Note: It wasn't really necessary to compute the CV Test MSE with random splitting because the `randomForest()` function computes the out of bag Test MSE using bootstrapping. It is a helpful way to compare actual data against predictions.

Now, let's change the number of bootstrapped trees, say `ntree = 25`.

```
bag.boston.25 <- randomForest(medv ~ .,
                              data = Boston.train,
                              mtry = 13,
                              ntree = 25)
bag.pred.25 <- predict(bag.boston.25,
                       newdata = Boston.test)
plot(bag.boston.25)
```




```
mean( (bag.pred.25 - Boston.test$medv) ^ 2 )
```

```
[1] 23.37267
```

Notice that the MSE is a bit higher than with 500 trees, although not by a lot.

Bagging Classification Tree Example

Let's use the **Carseats** data set, but using a binary outcome **High** when sales are greater than $8K

```r
library(randomForest)
library(ISLR)
set.seed(1)

High <- as.factor(ifelse(Carseats$Sales > 8, 1, 0)) # Convert to binary
Carseats <- data.frame(Carseats, High) # Add the High to the data set
```

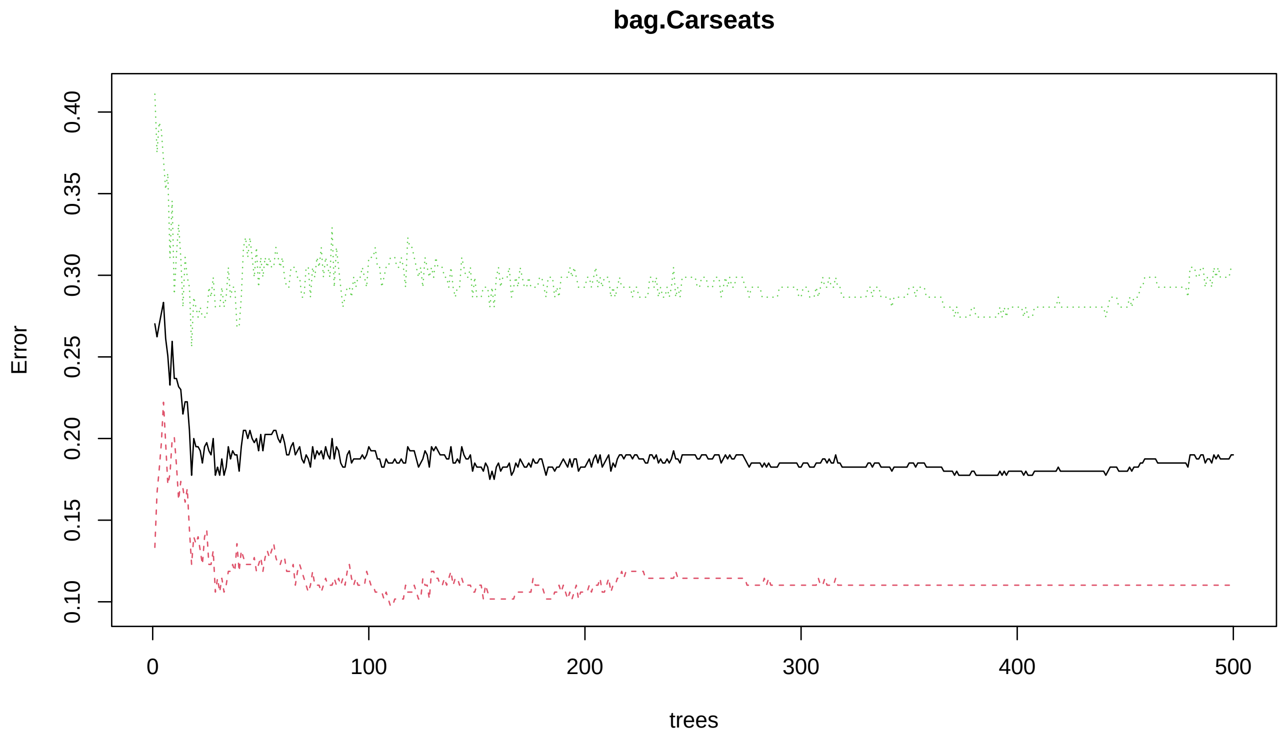
Now let's fit the BAagging tree

```r
bag.Carseats <- randomForest(High ~ . -Sales,
                             data = Carseats,
                             mtry = 10,
                             importance = T)
bag.Carseats
```

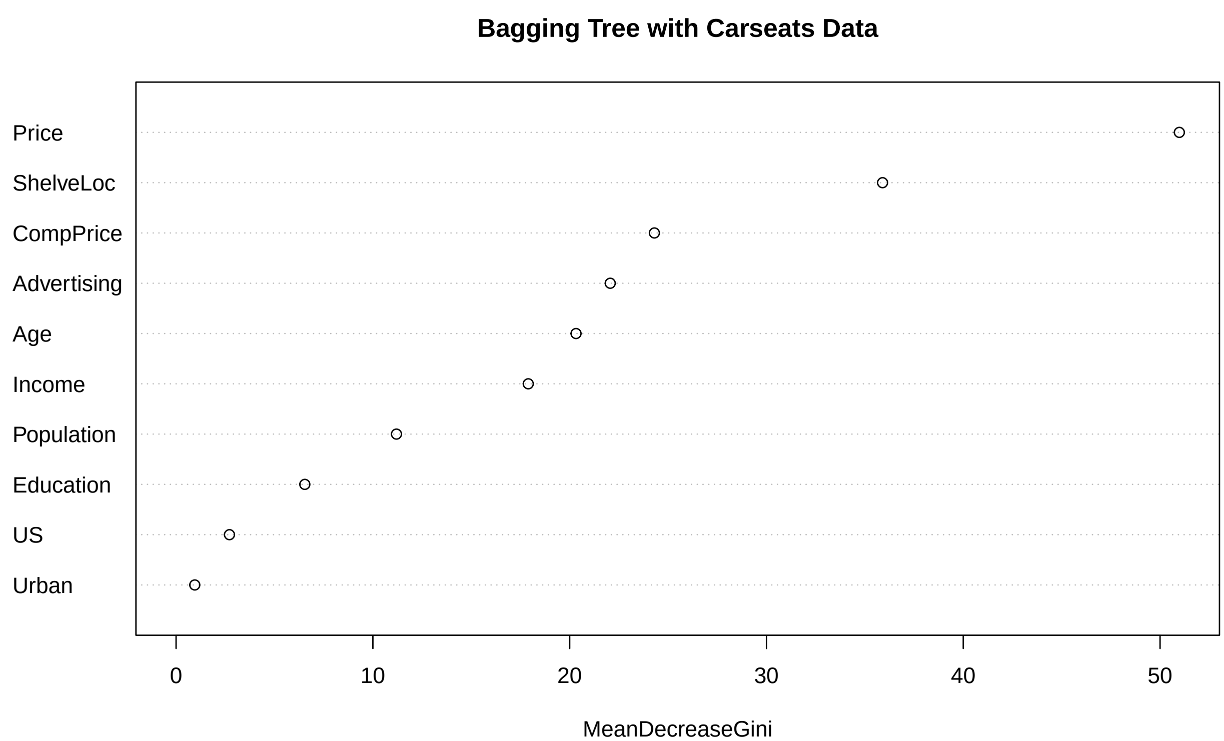
```
Call:
 randomForest(formula = High ~ . - Sales, data = Carseats, mtry = 10,      importance = T)
               Type of random forest: classification
                     Number of trees: 500
No. of variables tried at each split: 10

        OOB estimate of  error rate: 19%
Confusion matrix:
    0   1 class.error
0 210  26   0.1101695
1  50 114   0.3048780
```

CV and Variable Importance Plots

```r
plot(bag.Carseats)
```




```r
varImpPlot(bag.Carseats, type = 2,
           main = "Bagging Tree with Carseats Data")
```



```r
importance(bag.Carseats, type = 2)
```

```
            MeanDecreaseGini
CompPrice         24.3042930
Income            17.8955173
Advertising       22.0608417
Population        11.1980109
Price             50.9794826
ShelveLoc         35.8992613
Age               20.3232893
Education          6.5407075
Urban              0.9498763
US                 2.7116302
```

Random Forest Trees

The main difference with BAgging is that in Random Forest, the number of variables `m` used to fit the individual trees is a subset of all the available variables `p`, so `m <= p`. Bagging is a special case of Random Forest when m = p. Therefore, both methods use the same library and function `{randomForest}randomForest()`. The Random Forest method will select the first `m` predictors randomly, and then vary the chosen `m` predictors in each bootstrapped tree. This reduces the correlation between trees. The limitation of Bagging is that all trees are fitted with the same predictors, so results are likely to be somewhat similar or correlated. In contrast, Random Forest with `m < p` corrects for this issue because every tree will be different. After bootstrapping hundreds or thousands of trees, all predictors will be included at one time or another.

We use the same `randomForest()` function we used for bagging, but we specify a smaller number of sampled predictors with the parameter `mtry =` than the number of available predictors. The default is `mtry = p/3`, which is one third of the number of predictors in the model.

For the following illustration, we use the same function, train and test subsets we used for Bagging. I specify a Random Forest using trees with 6 random predictors each time. Everything else is the same as Bagging.

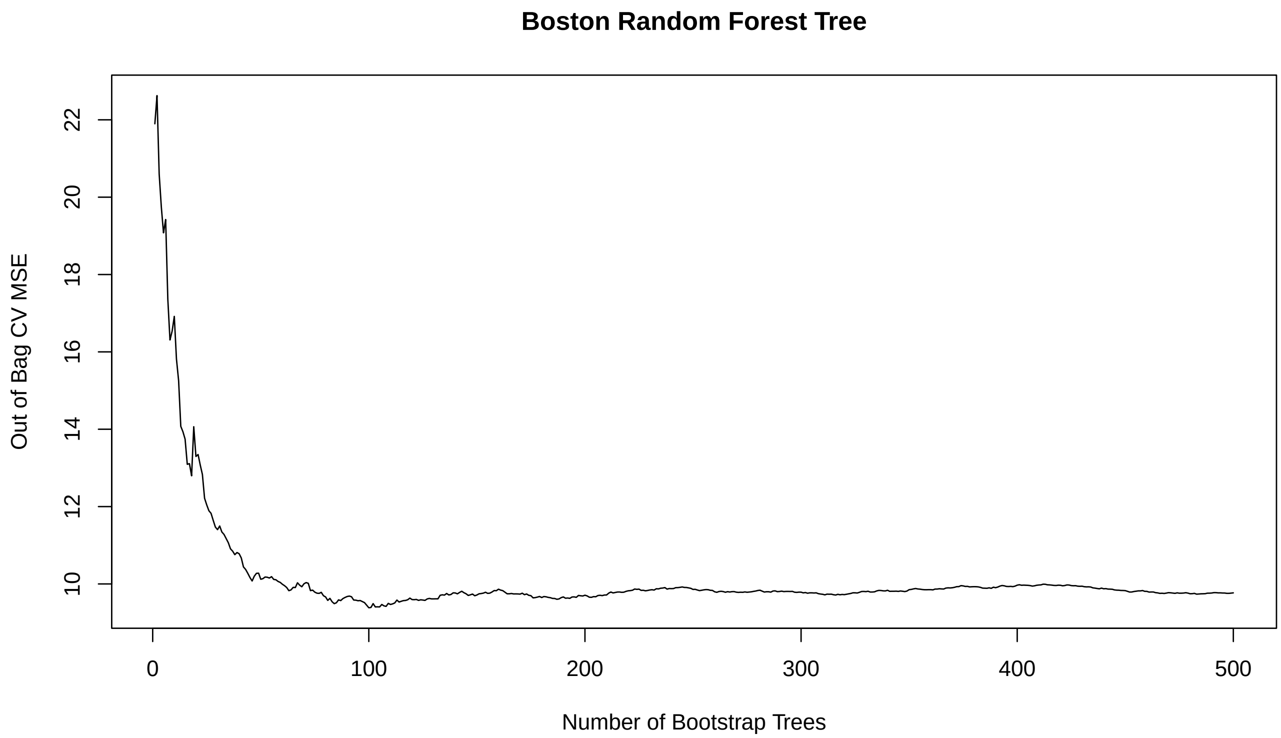
```r
library(randomForest) # if not loaded already
library(MASS) # For Boston housing data set, if not loaded already
rf.boston <- randomForest(medv ~ .,
                          data = Boston,
                          mtry = 6,
                          importance = T)

rf.boston # Inspect the results
```

```
Call:
 randomForest(formula = medv ~ ., data = Boston, mtry = 6, importance = T)
               Type of random forest: regression
                     Number of trees: 500
No. of variables tried at each split: 6

          Mean of squared residuals: 9.769129
                    % Var explained: 88.43
```

```r
cat("\n", "10FCV RMSE =", sqrt(mean(rf.boston)), "\n\n")
```

```
 10FCV RMSE = NA
```

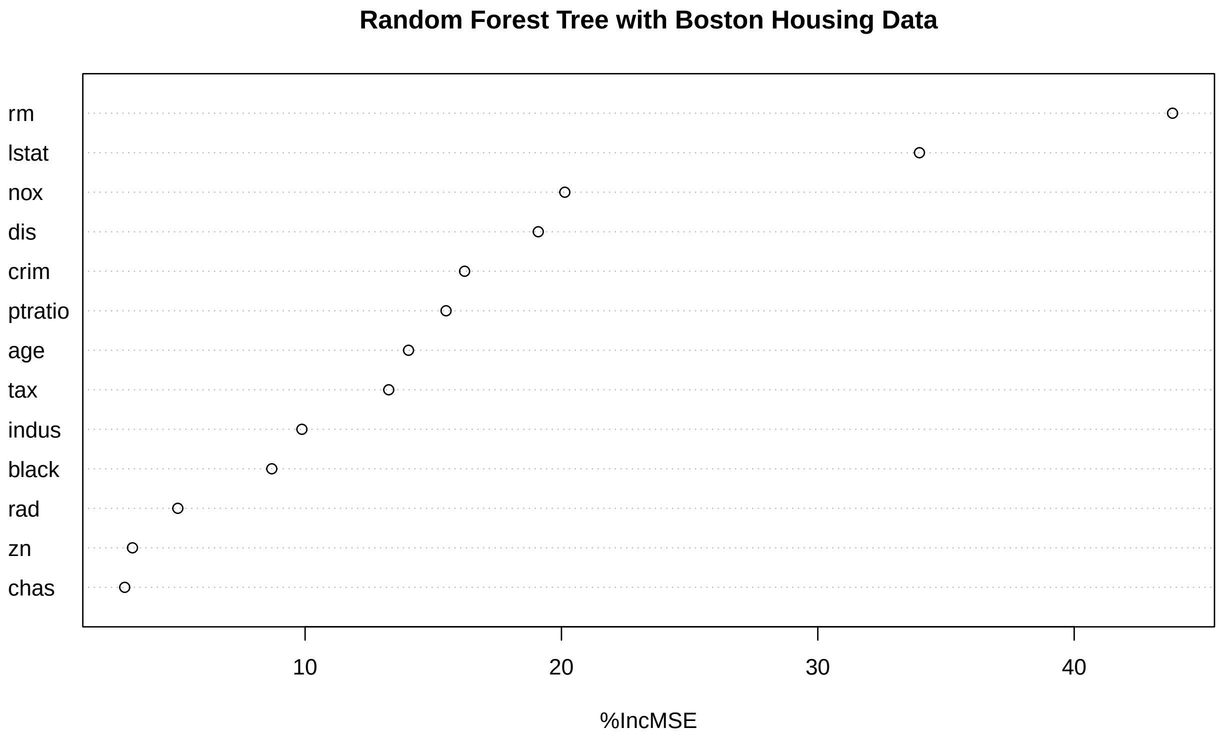
```r
plot(rf.boston$mse,
     main = "Boston Random Forest Tree",
     xlab = "Number of Bootstrap Trees",
     ylab = "Out of Bag CV MSE",
     type="l")
```



Variable Importance in Random Forest trees

It seems like 50 or 60 bootstrapped trees may be sufficient. Let's look at the resulting variable importance.

```r
varImpPlot(rf.boston,
           type = 1,
           main = "Random Forest Tree with Boston Housing Data")
```




```
round(importance(rf.boston,
                 type = 1),
      digits = 2) # To view the importance of each variable
```

```
        %IncMSE
crim      16.22
zn         3.27
indus      9.88
chas       2.96
nox       20.13
rm        43.84
age       14.03
dis       19.10
rad        5.03
tax       13.26
ptratio   15.50
black      8.70
lstat     33.96
```

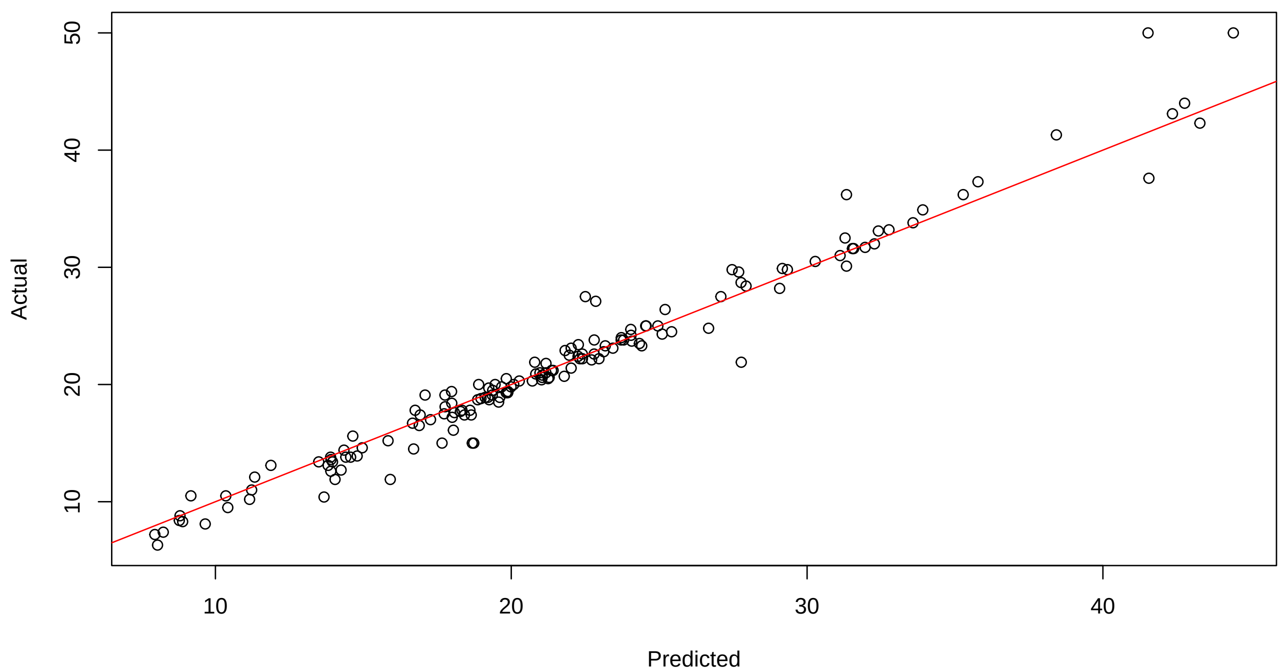
The results show that house size `rm` and overall community wealth `lstat` and are the most important predictors.

Random Forest Tree Predictions and Cross-Validation

Let's do predictions with the trained model and the test subset:

```
set.seed(1) # To get replicable results
trsize <- 0.7
```

```
train <- sample(1:nrow(Boston), trsize * nrow(Boston))
Boston.train <- Boston[train, ]
Boston.test <- Boston[-train, ]

rf.pred <- predict(rf.boston, newdata = Boston.test)

plot(rf.pred,
     Boston.test$medv,
     xlab = "Predicted",
     ylab = "Actual") # Plot pred vs. actual
abline(0, 1, col = "red") # 45 degree line (intercept = 0; slope = 1)
```



Let's now compute the CV Test MSE

```
mse.rf <- mean( (rf.pred - Boston.test $ medv) ^ 2 )
mse.rf
```

```
[1] 2.549496
```

Note that the MSE for this model is even smaller than for the Bagged model

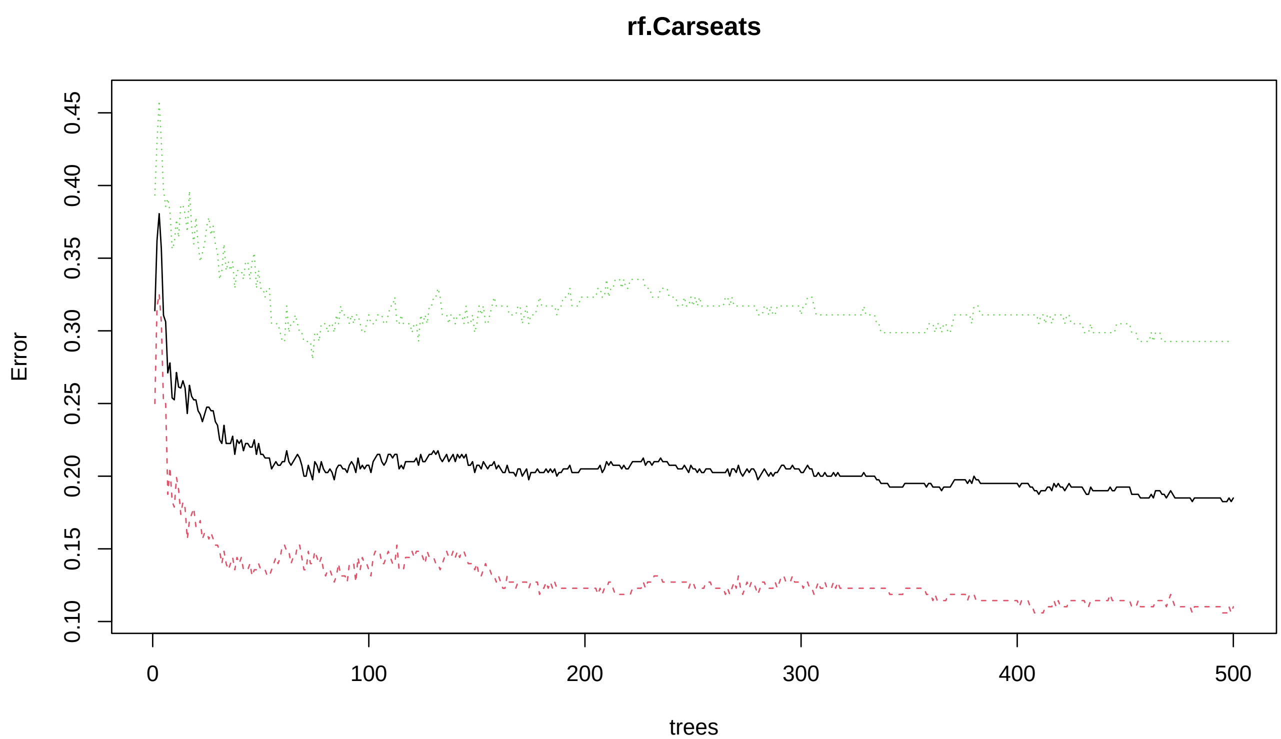
Random Foreset Classification Tree Example

```r
rf.Carseats <- randomForest(High ~ . -Sales,
                            data = Carseats,
                            mtry = 4,
                            importance = T)
rf.Carseats
```

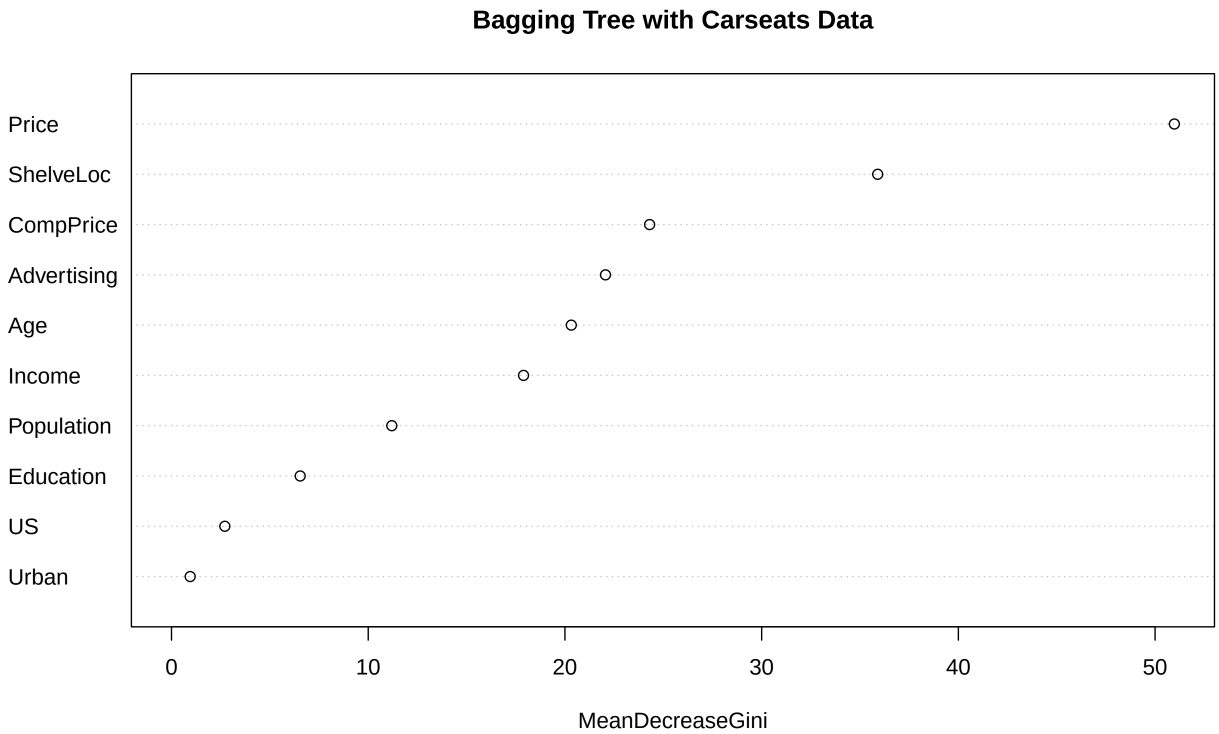
```
Call:
 randomForest(formula = High ~ . - Sales, data = Carseats, mtry = 4,      importance = T)
               Type of random forest: classification
                     Number of trees: 500
No. of variables tried at each split: 4

        OOB estimate of  error rate: 18.5%
Confusion matrix:
    0   1 class.error
0 210  26   0.1101695
1  48 116   0.2926829
```

CV and Variable Importance Plots

```r
plot(rf.Carseats) # The error flattens after about 70 bootstrapped trees
```




```r
varImpPlot(bag.Carseats, type = 2,
           main = "Bagging Tree with Carseats Data")
```




```r
importance(bag.Carseats, type = 2)
```

```
            MeanDecreaseGini
CompPrice         24.3042930
Income            17.8955173
Advertising       22.0608417
Population        11.1980109
Price             50.9794826
ShelveLoc         35.8992613
Age               20.3232893
Education          6.5407075
Urban              0.9498763
US                 2.7116302
```

Notice that we used the parameter `type = 2` to get the mean decrease in the **Gini** impurity.

Boosted Trees

For the boosted tree illustration, we use the `gbm()` function from the Generalized Boosted Models `{gbm}` library. We also use the **Boston** housing data set in the `{MASS}` library.

```r
library(gbm)
library(MASS)
```

Like Bagging and Random Forest, Boosting models fit several trees and aggregate the result. Unlike Bagging and Random Forest, Boosting does not fit several random bootstrapped trees, but it fits an initial tree with all the data, and then fits another one to explain and predict the residuals (errors), then again, etc.

Bagging and Random Forest are considered **fast** learning methods because the best model is generated in the first few samples and subsequent trees may or may not improve the MSE, whereas Boosting is considered to be a **slow** learning method because every new tree builds upon and improves upon the prior tree.

Boosted trees have a tuning parameter `lambda` (similar to shrinkage in Ridge and LASSO), which controls the speed of learning.

Aside: to understand this concept, imagine that you run an OLS regression or any other predictive model with certain predictors and you extract the residuals (i.e., errors). The residual values represent the portion of the outcome values that are not explained by the model. You can then build another regression model to explain (i.e., predict) those unexplained residuals. This new regression will explain some of the error variance, but will also yield new errors (smaller than the first ones, because some of the variance in the errors is already explained with the second model). Then you can fit a third regression model to explain the new residuals, and so on. You can then aggregate all the regression models, which on the aggregate, will have small residuals. Boosting applies this concept when generating trees.

When fitting boosted trees, we use the parameter:

- `distribution = "gaussian"` (i.e., normal distribution) for regression trees, and
- `distribution = "bernoulli"` for classification trees

Let's fit a model with 5000 trees, limiting the depth of each tree to 4 and using all available predictors

```
set.seed(1) # To get replicable results

boost.boston <- gbm(medv ~ .,
                    data = Boston,
                    distribution = "gaussian", # Quantitative outcome
                    shrinkage = 0.01, # Lambda
                    cv.folds = 10, # 10FCV
                    n.trees = 5000, # Number of boosted trees
                    interaction.depth = 4) # 4 tree splits

boost.boston # Display the basic model data
```

```
gbm(formula = medv ~ ., distribution = "gaussian", data = Boston,
    n.trees = 5000, interaction.depth = 4, shrinkage = 0.01,
    cv.folds = 10)
A gradient boosted model with gaussian loss function.
5000 iterations were performed.
The best cross-validation iteration was 3234.
There were 13 predictors of which 13 had non-zero influence.
```

```
cat("\n", "RMSE =", sqrt(mean(boost.boston$cv.error)), "\n")
```

```
 RMSE = 3.413179
```

Let's find the number of boosted trees with smallest CV Test Error

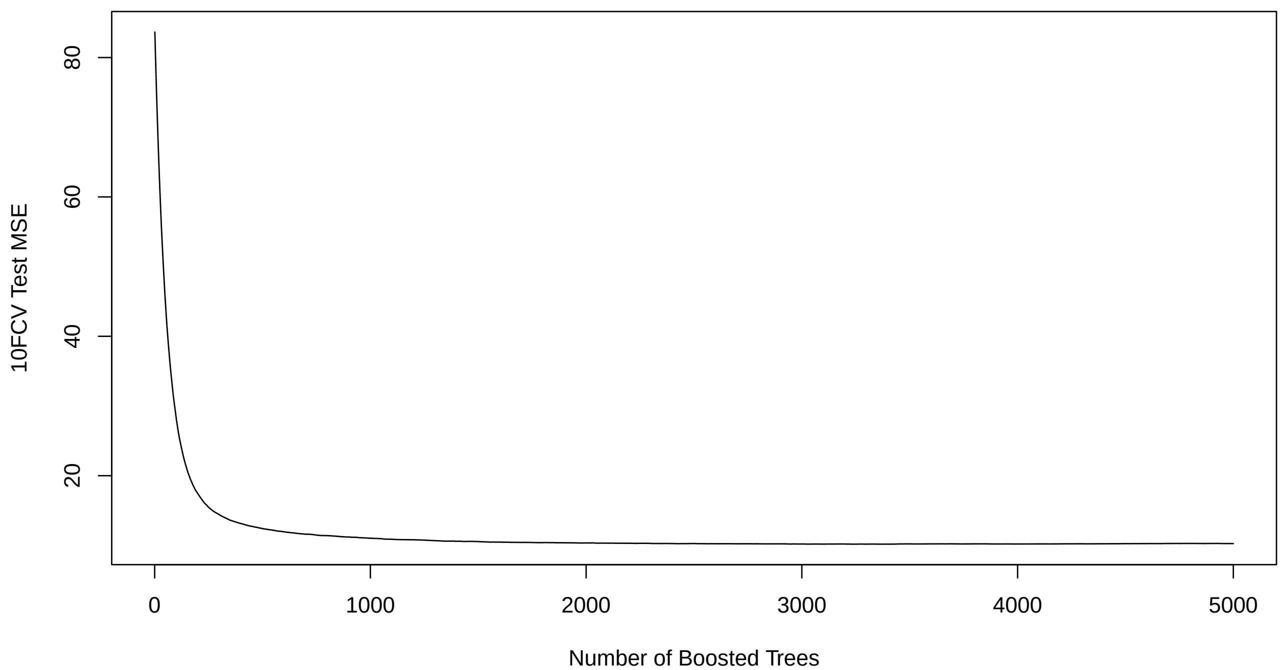
```
best.num.trees <- which.min(boost.boston$cv.error) # Which tree
min.10FCV.error <- round(min(boost.boston$cv.error), # Smallest CV Test Error
                         digits = 4)
paste("Min 10FCV Test Error =", min.10FCV.error,
      "at", best.num.trees, "trees")
```

```
[1] "Min 10FCV Test Error = 10.1746 at 3234 trees"
```

Boosted Tree Plots

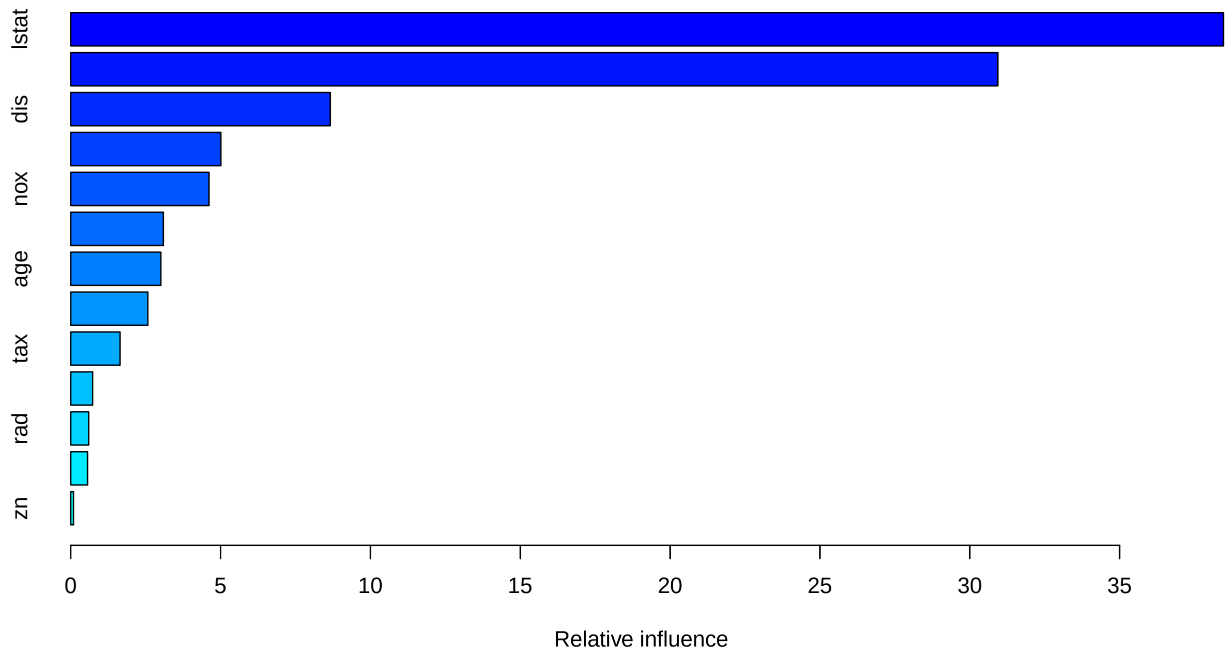
Let's inspect how the number of trees affects the CV Test MSE

```
plot(boost.boston$cv.error,
     type = "l",
     xlab = "Number of Boosted Trees",
     ylab = "10FCV Test MSE")
```



Let's plot the variable importance of the predictors. This graph shows the relative influence on the outcome of each predictor. The score is normalized, so that they add up to 100.

```
summary(boost.boston)
```




```
            var     rel.inf
lstat     lstat 38.46537753
rm           rm 30.93608309
dis         dis  8.66084763
crim       crim  5.01123220
nox         nox  4.61559339
ptratio ptratio  3.09149607
age         age  3.00897831
black     black  2.57458355
tax         tax  1.64718724
indus     indus  0.73181333
rad         rad  0.60145255
chas       chas  0.56215998
zn           zn  0.09319513
```

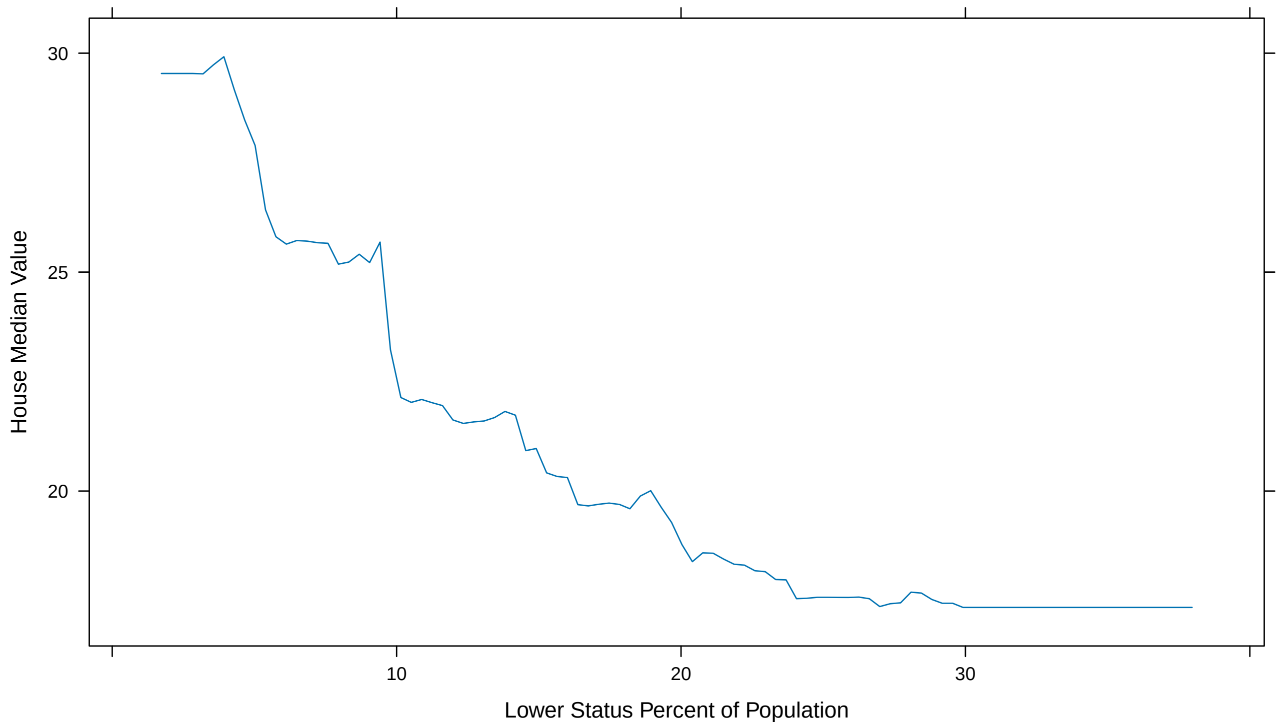
Note that `lstat` and `rm` are the most important variables, just like with Bagging and Random Forest.

Boosted Trees and Partial Dependencies

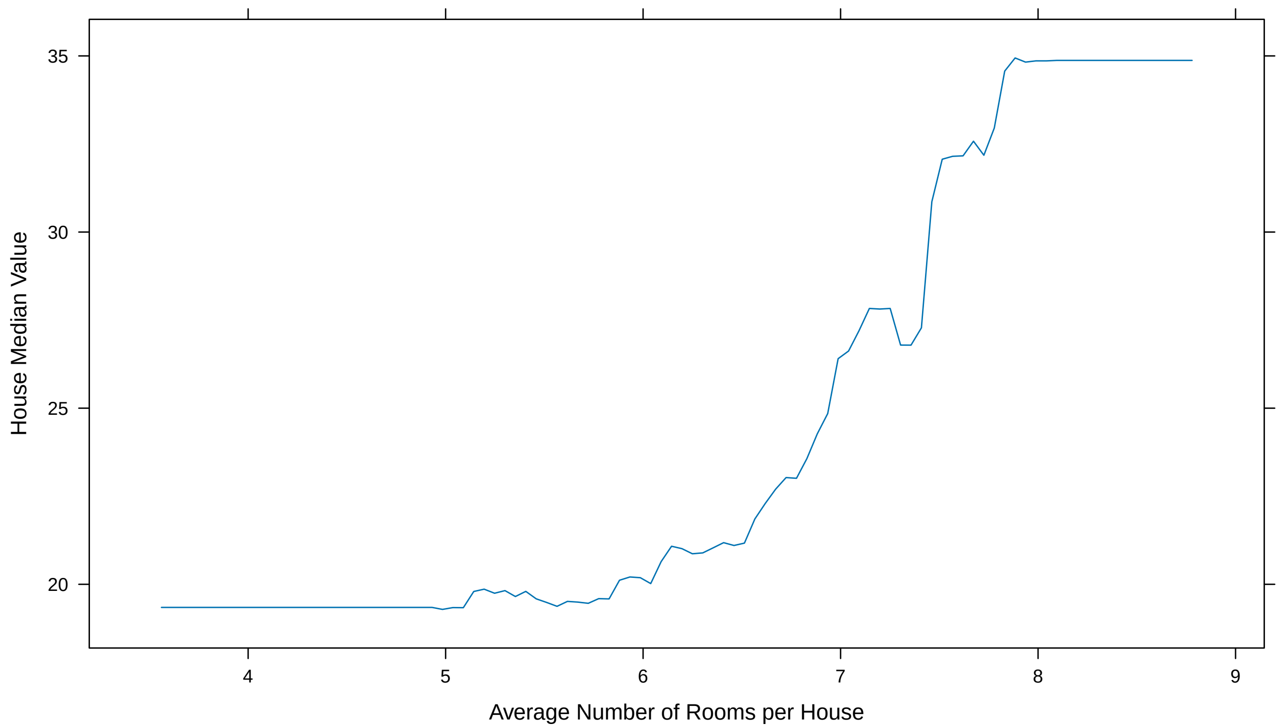
Partial dependencies show how a particular predictor affects the outcome variable, holding other predictors constant. It is important to note that this is NOT an effect. You can derive effects from the graph, but an effect is how much Y increases when X increases by 1. In contrast, in the partial dependencies graph, we can see what is the value of Y for a given value of X, holding everything else constant.

To see how predicted house median values in Boston `medv` vary with `lstat` and `rm`, we can use the `i` vector. The `boost.boston` object has an attribute property called `i`, which is a vector containing the variables used to build the model. To plot how the outcome variable is partially affected by a predictor use the parameter `i =` to specify the predictor of interest.

```r
plot(boost.boston,
     i = "lstat",
     ylab = "House Median Value",
     xlab = "Lower Status Percent of Population")
```




```r
plot(boost.boston,
     i = "rm",
     ylab = "House Median Value",
     xlab = "Average Number of Rooms per House")
```



Boosted Tree Predictions and Cross-Validation

Let's compare results across models using the same random sample split

```
trsize <- 0.7
train <- sample(1:nrow(Boston),
                trsize * nrow(Boston)) # Train index
set.seed(1) # For replicable results
Boston.train <- Boston[train, ] # Train subset
Boston.test <- Boston[-train, ] # Test Subset

boost.boston.tr <- gbm(medv ~ .,
                       data = Boston.train,
                       distribution = "gaussian",
                       shrinkage = 0.01, # Lambda
                       cv.folds = 10, # 10FCV
                       n.trees = 5000, # Number of boosted trees
                       interaction.depth = 4) # 4 tree splits
boost.boston.tr # Basic output
```

```
gbm(formula = medv ~ ., distribution = "gaussian", data = Boston.train,
    n.trees = 5000, interaction.depth = 4, shrinkage = 0.01,
    cv.folds = 10)
A gradient boosted model with gaussian loss function.
5000 iterations were performed.
The best cross-validation iteration was 4978.
There were 13 predictors of which 13 had non-zero influence.
```

```r
boost.pred <- predict(boost.boston.tr, # Predict with the trained model
                      newdata = Boston.test, # Test with the test subset data
                      n.trees = 5000)

mse.boost <- mean( (boost.pred - Boston.test$medv) ^ 2) # MSE
mse.boost
```

```
[1] 11.77837
```

Let's compare results. Which method is better

```r
round(cbind("MSE Plain Tree" = mse.tree,
            "MSE Bagging" = mse.bag,
            "MSE Random Forest" = mse.rf,
            "MSE Boosted Tree" = mse.boost),
      digits = 3)
```

```
     MSE Plain Tree MSE Bagging MSE Random Forest MSE Boosted Tree
[1,]         36.232       2.842             2.549           11.778
```

In this example, the boosted tree model is far superior to the other 3 models, and the plain tree is far inferior to the other 3 models. Bagging and Random Forest are somewhere in between.

Shrinkage

Boosting has a similar **shrinkage** effect, just like Ridge and LASSO regression. The difference is that Ridge and LASSO shrink the regression coefficients, whereas boosting shrinks earlier outcome predictions to weaken the model, to then be further strengthened as it learns from the errors in subsequent models. The shrinkage applies over each tree model, including the first one. A small lambda shrinks he prior tree model predictions more, thus making the early predictions less important for the final aggregated model (i.e., slow learning). Large lambdas give more weight to the initial trees, thus learning fast.

To vary the shrinkage factor lambda, set the `shrinkage =` parameter. The default is `0.01`. The default for the interaction tree depth is 1. Let's lower the shrinkage parameter and retain the depth of 1 knot (i.e., a tree "stomp")

```r
shrk <- 0.001 # Let's try a slow learning rate
dpth <- 1 # interaction tree depth

boost.boston.shrk <- gbm(medv ~ .,
                         data = Boston.train,
                         distribution = "gaussian",
                         n.trees = 5000,
                         interaction.depth = dpth,
                         shrinkage = shrk,
                         verbose = F)
```

Now let's do predictions with the test data

```r
medv.boost.pred <- predict(boost.boston.shrk,
                           newdata = Boston.test,
                           n.trees = 5000)
mean( (medv.boost.pred - Boston.test$medv) ^2)
```

```
[1] 18.92473
```

Not much better

Boosted Classification Tree

Classification trees are fitted similarly to regression boosted trees, but with two important differences:

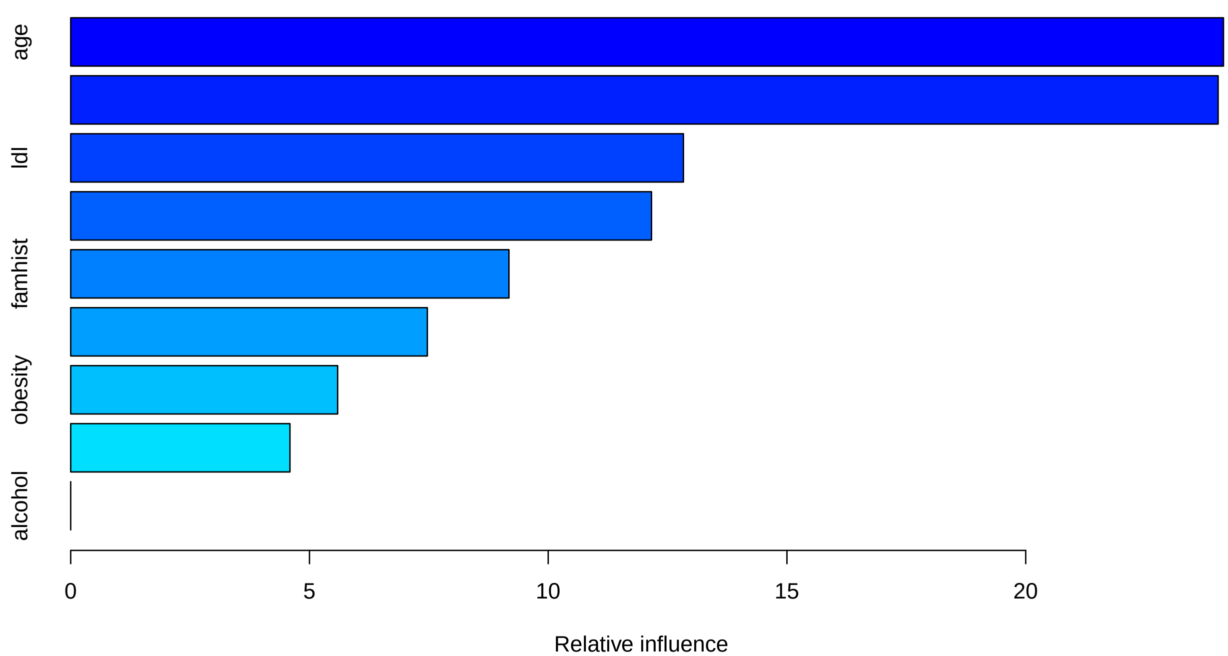
- Need to use the parameter `distribution = "bernoulli"` instead of `"gaussian"`
- Need to convert the outcome from a factor variable to 0-1 numeric

For example, using the Heart.csv data set to predict `chd` (coronary heart disease). Let's read the data again, for convenience

```
heart <- read.table("Heart.csv",
                    sep = ",",
                    header = T,
                    stringsAsFactors = T)
```

We can now fit a classification boosted tree

```
boost.chd <- gbm(chd ~ . ,
                 data = heart,
                 distribution = "bernoulli")
summary(boost.chd)
```

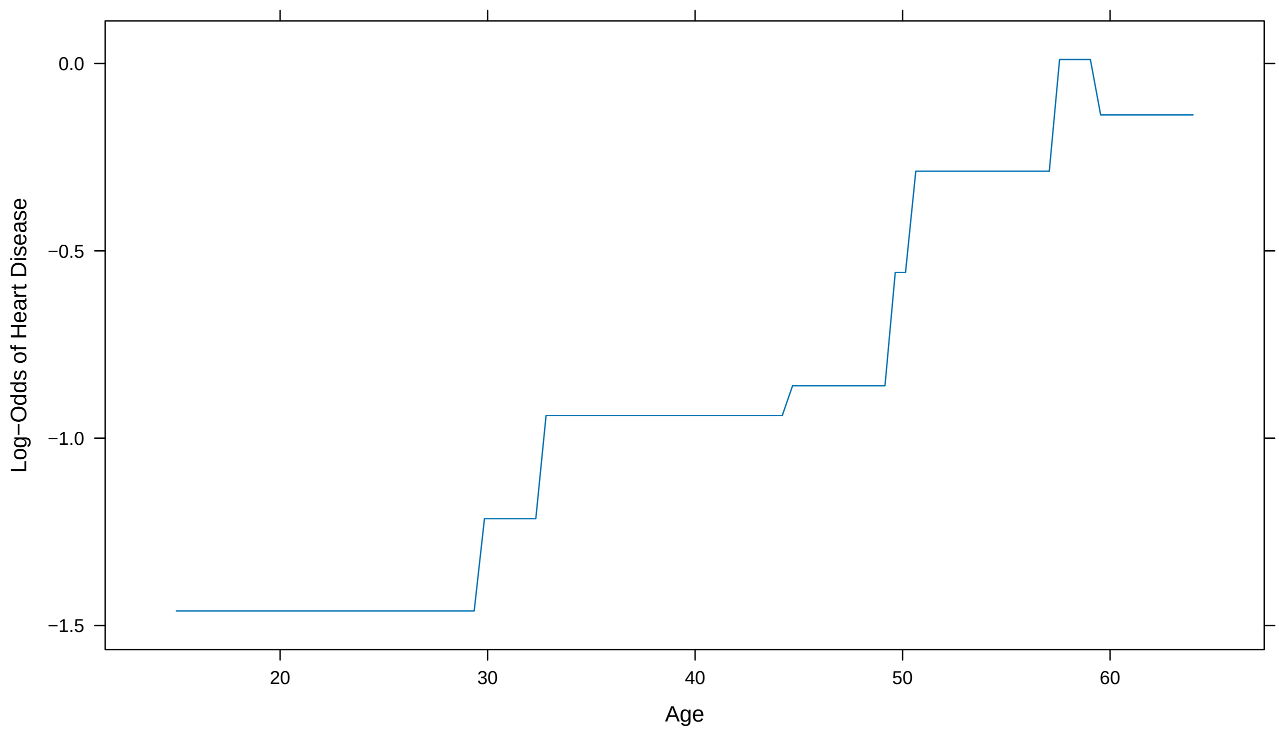



```
                var   rel.inf
age             age 24.142084
```

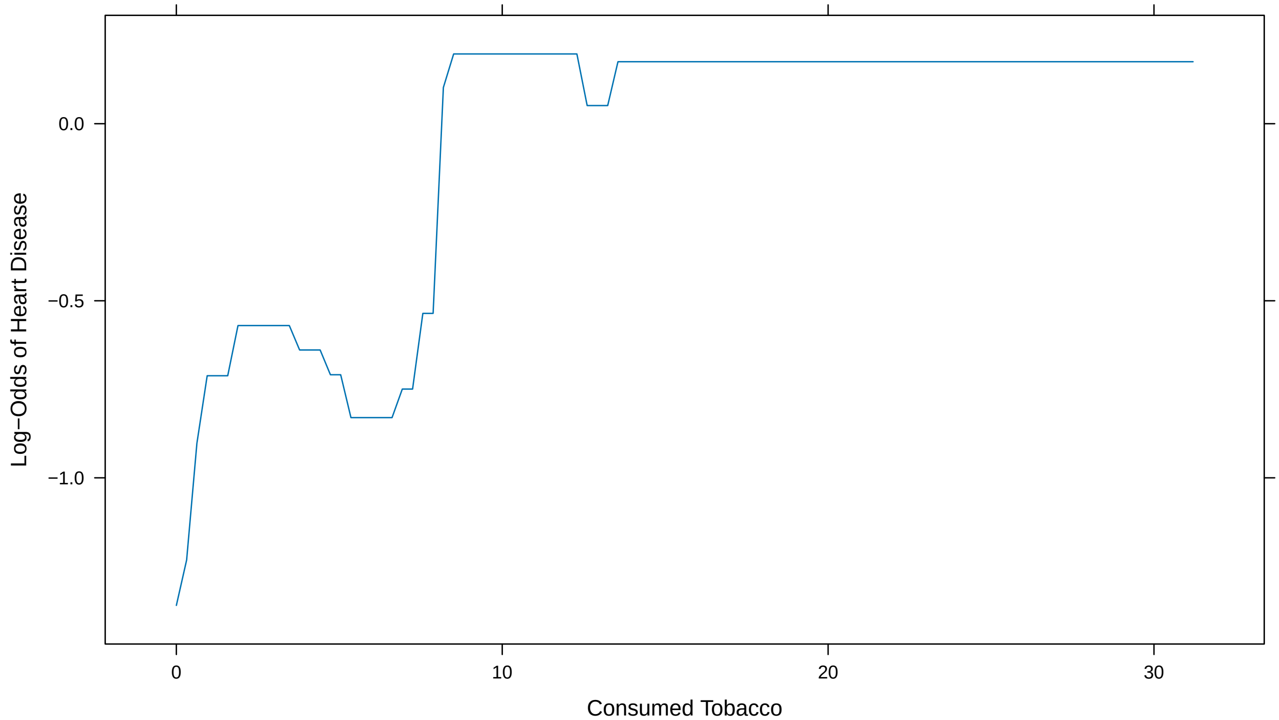
```
tobacco      tobacco 24.030552
ldl              ldl 12.832443
typea          typea 12.164290
famhist      famhist  9.178191
adiposity adiposity  7.469208
obesity      obesity  5.591556
sbp              sbp  4.591675
alcohol      alcohol  0.000000
```

The predictors `age` and `tobacco` are the most important predictors of heart disease. Let's plot their partial correlation. The **Y axis** in the plot below depicts the **Log-Odds** of `chd = 1`. To convert to odds use `exp(Log-Odds)`.

```
plot(boost.chd,
     i = "age",
     ylab = "Log-Odds of Heart Disease",
     xlab = "Age")
```




```
plot(boost.chd,
     i = "tobacco",
     ylab = "Log-Odds of Heart Disease",
     xlab = "Consumed Tobacco")
```



For example, the Log-Odds of having coronary heart disease at the age of 60 is about - 0.2, holding everything else constant. The odds are exp(-.02) - 0.82. The probability is Odds / (Odds + 1) = 0.45 or 45%

```
odds <- round(exp(-0.2), digits = 3)
paste("The odds of coronary heart disease are", odds)
```

```
[1] "The odds of coronary heart disease are 0.819"
```

```
prob <- round(odds / (1 + odds), digits = 3)
paste("The probability of coronary heart disease are", prob)
```

```
[1] "The probability of coronary heart disease are 0.45"
```